CONFIDENTIAL · INTERNAL

Product Design Specification

Draft v1.0 · May 2026

00 — Overview

Vision

Governed AI analytics for any regulated application

The **AI Analytics Platform** enables any application — and any AI agent operating within that application — to access governed, explainable, role-aware analytical intelligence against large-scale datasets, without building semantic query infrastructure. The experience is modelled on the best institutional analytics environments: deterministic metric resolution, auditable execution, role-enforced data access, and AI-generated insights that are always anchored to governed, registered business definitions.

It is not a general-purpose analytics tool and not a natural-language SQL interface. Each deployment is a **governed analytical specialist** configured to its host application's metric domain, entitlement model, and regulatory context.

The platform serves multiple consumer types simultaneously: human users querying through an embedded analytics component, conversational AI users asking questions through the **AI Chat Platform**, and autonomous agents running scheduled or event-triggered analytical workflows. All consumers share the same governance pipeline and lineage trail.

> **Governing intent:** Give any application team the ability to deploy a production-grade, governed AI analytics layer into their product within days — fully scoped to their semantic metric domain, connected to their execution engines via a federated query planner, enforcing their entitlement model, and producing a complete analytical lineage trail for every query — whether that query originates from a human user, a conversational assistant, or an autonomous agent.

What the platform is and is not

It is	It is not
A headless MCP API service — it returns structured JSON result sets and optional Vega-Lite chart specifications; any application or agent can consume it regardless of their UI stack	A general-purpose SQL query interface, BI tool replacement, or UI-rendering layer
A platform where all AI-accessible metrics are registered, governed, and version-controlled in a Semantic Metrics Registry	A system that allows LLMs to generate arbitrary SQL against physical schemas
A federated query engine that routes governed analytical plans to appropriate execution backends	A single-engine analytics layer coupled to one database or warehouse
A role-aware entitlement enforcement layer applied at the semantic tier before execution	A system that relies on database-level access controls as the primary security boundary for AI queries

It is	It is not
A headless chart specification layer — the Visualisation Ontology selects chart contracts and produces Vega-Lite specs; rendering is always the consumer's responsibility	A system that allows LLMs to select chart types ad hoc without a governing contract, or that renders charts directly
A complete analytical lineage trail from business question to execution result	A black-box analytics system with no explainability mechanism
A governed narrative synthesis layer anchored to registered metric values	A free-text generation layer that can introduce metric values not present in the execution result
An MCP Capability Layer accessible to any MCP-compatible AI orchestrator — conversational assistants, autonomous agents, and pipelines alike	A data API accessible only to a specific AI platform or a specific consumer type

Guiding principles

Principle	What it means in practice
Governed by default	No metric, query, or result escapes the governance pipeline. Circuit breakers, compliance classification, cost controls, and role enforcement apply to every request regardless of consumer type. There is no fast path that bypasses governance.
Semantic, not syntactic	All analytical intent is expressed in business vocabulary registered in the SMR. Physical schemas, SQL, and execution-engine specifics are never exposed to AI models or to consumers. The platform owns the translation from intent to execution.
Headless and consumer-agnostic	The platform produces structured JSON results and Vega-Lite chart specifications. It never renders charts or generates HTML. Rendering is always the consumer's responsibility — the `<ai-analytics>` component, the AI Chat Platform, the `vite2img` service, or any Vega-Lite-compatible library.
Role enforcement at the semantic tier	Entitlements are applied at the Role-Aware Projection Layer before any Logical Query Plan is compiled. The execution engine is not the security boundary — a query that the user is not entitled to see never reaches physical execution.
Lineage for everything	Every analytical result carries a `result_id` linking to a complete, queryable provenance chain from natural-language intent through SMR resolution, role projection, plan compilation, engine routing, and result assembly.
Equal governance for all consumer types	Human users, conversational assistants, and autonomous agents route through the same pipeline and receive results with identical metric definitions, entitlement enforcement, and lineage provenance. There is no privileged consumer class and no governance bypass.

Scope

In scope

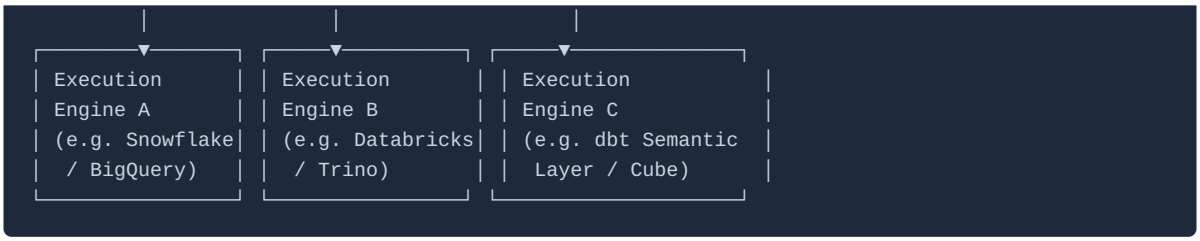
- Headless MCP API (`POST /v1/mcp`) — the sole entry point for all consumers; returns structured JSON result sets and optional Vega-Lite chart specifications; no rendering layer
- Multi-tenant architecture with complete per-tenant data isolation
- Semantic Metrics Registry (SMR) — the governing catalogue of all resolvable metrics, dimensions, hierarchies, aggregation rules, ownership assignments, and lineage metadata
- Analytics DSL — a constrained, AI-readable query language that exposes business semantics and compiles to engine-agnostic Logical Query Plans
- Federated Query Planner — routes Logical Query Plan fragments to registered execution engines and assembles results
- Role-Aware Projection Layer — applies entitlement filters (row restrictions, column masks, metric visibility rules) at the semantic tier before plan compilation
- Semantic Execution Governance — circuit breakers, cost controls, query complexity limits, and compliance classification checks applied before any physical engine call
- Visualisation Ontology — a governed schema of chart contracts, interaction semantics, drilldown definitions, and chart contract parameters; produces Vega-Lite specifications returned to the consumer — the platform does not render charts
- MCP Capability Layer — exposes bounded, pre-defined analytical operations to AI orchestrators via MCP-compatible interfaces
- Narrative synthesis — LLM-generated prose explanations anchored to execution results, governed to prohibit metric hallucination
- Analytical lineage trail — complete, queryable lineage from intent resolution through semantic planning, execution, and result delivery
- Financial services reference semantic model — pre-built metric definitions for wealth management, banking, investment management, and regulatory reporting domains
- Host-configured analytical domain scoping, metric access policies, and execution engine registration
- Governed drilldown — traversal of registered analytical hierarchies within host-configured scope

Out of scope

- Chart or table rendering of any kind — consumers are responsible for all display
- Web component or UI embedding layer — any such component is a separate consumer product
- Direct SQL execution against host databases
- Physical schema exposure to AI model context
- Ad hoc LLM-generated SQL at any layer
- Real-time streaming data ingestion (v1)
- General-purpose BI authoring beyond governed narrative synthesis
- Cross-tenant metric federation
- Unauthenticated analytical access

Platform architecture


Consumer (any MCP-compatible caller)
AI Chat Platform · autonomous agent · custom application
— presents user JWT with role claims
POST /v1/mcp (JWT + MCP tool call)
AI Analytics Platform
Semantic Intent Layer
(LLM resolves natural language → analytical intent)
Semantic Metrics Registry (SMR)
(governed metric definitions, dimensions, hierarchies,
aggregation rules, lineage, ownership, access policies)
Role-Aware Projection Layer
(entitlement enforcement — rows, columns, metrics)
Analytics DSL Compiler
(governed DSL → Logical Query Plan)
Semantic Execution Governance
(circuit breakers, cost limits, compliance classification)
Federated Query Planner (FQP)
(routes LQP fragments to registered execution engines)
Visualisation Ontology
(deterministic chart contract selection; Vega-Lite spec
generation — no rendering; spec returned to consumer)
Narrative Synthesis Engine
(governed LLM prose anchored to execution result values)
Analytical Lineage Store
(intent → plan → execution → result, queryable)



Components

Semantic Intent Layer receives natural language from the user or AI orchestrator, resolves it against the SMR, and produces a structured analytical intent representation — the set of metrics, dimensions, filters, and hierarchy traversals being requested.

Semantic Metrics Registry (SMR) is the governing catalogue of all resolvable analytical concepts. It defines what can be queried, how metrics are computed, what dimensions are permissible, and who owns each definition. It is the single source of truth for metric semantics across the platform.

Role-Aware Projection Layer applies the authenticated user's entitlement claims against the resolved intent. It filters the metric set to what the user is permitted to see, injects row-level security predicates, and applies column-level masking rules — before any query plan is compiled.

Analytics DSL Compiler translates the projected analytical intent into an engine-agnostic Logical Query Plan (LQP) expressed in the platform's Analytics DSL. The LQP is a directed acyclic graph (DAG) of analytical operations with no physical engine references.

Semantic Execution Governance validates the LQP against circuit breakers (cost estimates, complexity limits, data classification compliance) before releasing it to the Federated Query Planner.

Federated Query Planner (FQP) decomposes the LQP into engine-specific sub-plans, routes them to registered execution engines, handles result assembly, and manages caching and materialisation.

Visualisation Ontology receives the assembled result set and applies the governing Visualisation Ontology to select a chart contract and parameterise it from the result schema. It produces a **Vega-Lite specification** — a structured JSON chart description — that is returned to the consumer as part of the MCP tool response. The Analytics Platform does not render charts; rendering is always the consumer's responsibility (e.g., the `<ai-analytics>` web component, the AI Chat Platform's Vega-Lite renderer, or the optional `vite2img` service for static image generation).

Narrative Synthesis Engine generates governed prose explanations of the result, anchored exclusively to the values present in the execution result — LLM hallucination of metric values is architecturally prohibited.

Analytical Lineage Store persists the complete chain from intent to result for every query, providing explainability and regulatory audit support.

Headless by design

The AI Analytics Platform is a **headless service**. It has no rendering layer and produces no HTML, SVG, or rendered UI output of any kind. Every analytical request returns a structured MCP tool response containing:

Output field	Type	Description
`result_id`	string	Unique identifier for this execution result, linking to the full lineage record
`display_spec`	JSON object	A JSON display specification — always present. Either a standard Vega-Lite chart spec or a platform-defined tabular spec (see below). Consumers render from this object regardless of result shape.
`narrative`	string (optional)	Governed prose explanation produced by the Narrative Synthesis Engine — present when the host has enabled narrative synthesis for the tenant

Display spec format

Vega-Lite specifications are pure JSON, which makes it natural to use a single `display_spec` envelope for all result types. The platform returns one of two JSON shapes, distinguished by a `type` field:

Chart result — a standard Vega-Lite specification:

```
{
  "type": "vega-lite",
  "$schema": "https://vega.github.io/schema/vega-lite/v5.json",
  "mark": "bar",
  "data": { "values": [ ... ] },
  "encoding": {
    "x": { "field": "portfolio", "type": "nominal" },
    "y": { "field": "tracking_error", "type": "quantitative" }
  }
}
```

Tabular result — a minimal platform extension using the same JSON envelope:

```
{
  "type": "table",
  "columns": [
    { "field": "portfolio",      "label": "Portfolio",      "type": "string"  },
    { "field": "tracking_error", "label": "Tracking Error", "type": "number", "format": ".
2%" },
    { "field": "limit",          "label": "Limit",          "type": "number", "format": ".
2%" },
    { "field": "breached",       "label": "Breached",       "type": "boolean" }
  ],
  "data": [ ... ]
}
```

The `type: "table"` spec is a deliberate, minimal extension of the Vega-Lite JSON convention — it carries the same `data` array and typed column definitions, but signals to the consumer that a grid layout is the appropriate presentation rather than a chart. Vega-Lite itself has no native table mark; this extension fills that gap without introducing a second output format.

The Visualisation Ontology determines which shape is returned based on the result schema and the registered chart contract for the query type. Consumers always render from `display_spec.type` — they do not need to inspect the raw result to decide how to display it.

Rendering is always the consumer's responsibility:

Consumer	How they render
Custom analytics UI (host-built)	Inspect `display_spec.type`; render `"vega-lite"` with any Vega-Lite-compatible library (Vega-Embed, etc.) and `"table"` with their own grid component
AI Chat Platform	Native content rendering pipeline — `"vega-lite"` via the built-in Vega-Lite renderer; `"table"` via the built-in data table renderer
Agentic consumers producing PDF/email reports	Pass `display_spec` to the optional `vite2img` service, which handles both `"vega-lite"` (renders to PNG/SVG) and `"table"` (renders to an HTML table image)
Custom consumers	Any Vega-Lite library for chart specs; any grid/table component for the tabular extension

The optional **`vite2img` service** accepts a `display_spec` JSON object and returns a static image (PNG or SVG). It is a separate, independently deployable service intended for use cases where interactive rendering is unavailable — automated report generation, email delivery, PDF production, and batch pipelines. It does not interact with the Analytics Platform's governance pipeline.

This headless design means: - The Analytics Platform can serve any consumer regardless of their UI stack. - A single `display_spec` field is always present — consumers never need to branch on whether a chart or table was produced. - The governed chart contract (chart type, axis semantics, colour by meaning) is enforced at spec-generation time — rendering libraries cannot override it.

Dependencies

Dependency	Role
AI provider	Provider-agnostic abstraction used by the Semantic Intent Layer and Narrative Synthesis Engine. The platform maps tiers to the tenant's configured provider's current models.
Platform storage	Relational database with RLS for SMR records, lineage records, and configuration; object storage for cached result sets and artefacts.
Platform edge function	JWT handling, intent resolution API, DSL compilation, governance checks, FQP orchestration, result assembly, Vega-Lite spec generation.
Host authentication	The host application issues JWTs for its users, including role claims used by the Role-Aware Projection Layer.
Host execution engines	The host's registered analytical backends (data warehouses, semantic layers, OLAP engines) that execute physical query plans.

Dependency	Role
AI Chat Platform	The primary conversational consumer of the Analytics Platform's MCP Capability Layer. See Role in the AI-Enablement Product Ecosystem below.
`vite2img` static image service	Optional complementary service — accepts a Vega-Lite specification and returns a static PNG or SVG. Used by agentic consumers, report pipelines, and email delivery workflows where interactive rendering is unavailable.
Semantic Registry Service	Complementary ecosystem service — a curated library of pre-built metric definitions for financial services domains.
Regulatory Reference Service	Complementary ecosystem service — regulatory metric definitions for compliance reporting (Basel III/IV, IFRS 9, MiFID II, etc.).
Benchmark Data Service	Complementary ecosystem service — market benchmark and index data integrated as dimensional reference data.

Role in the AI-Enablement Product Ecosystem

Two-product architecture — and beyond

The AI Analytics Platform and the **AI Chat Platform** form a complementary two-layer AI-enablement offering. They are designed to be deployed together, with clear and deliberate division of responsibility:

Layer	Product	Responsibility
Conversational surface	AI Chat Platform	Generative chat interface, conversation management, multi-modal content rendering, tool call transparency, shared conversations, memory, and audit trail. Provides the user's entry point for all interaction. Has no built-in analytical capability.
Analytical backend	AI Analytics Platform	Governed semantic metric resolution, large-scale federated query execution, role-aware entitlement enforcement, Vega-Lite chart specification generation, and lineage-backed result delivery. Headless — returns structured data and chart specs; has no rendering layer and no conversational surface.

The AI Chat Platform is **one consumer** of the Analytics Platform's capabilities, not the only one. The Analytics Platform's MCP Capability Layer (see 08-mcp-capability-layer.md) is an MCP-compatible interface designed to be consumed by any AI orchestrator — the AI Chat Platform today, and additional agentic consumers in the future.

> **Architectural intent:** Any AI agent that needs to answer quantitative questions against large datasets — whether a conversational assistant, an autonomous monitoring agent, a report-generation pipeline, or a compliance review workflow — should route through the Analytics Platform's MCP Capability Layer rather than building its own data access path. The governance pipeline, role enforcement, and lineage trail apply equally to all consumers, regardless of how they invoke the capabilities.

This means the Analytics Platform's value grows with the number of AI consumers an organisation operates. As the AI ecosystem matures — multi-agent workflows, scheduled analytical agents, event-triggered compliance monitors — each new agentic consumer gets the same governed, lineage-backed analytical foundation without re-implementing query governance independently.

This separation is intentional. The AI Chat Platform remains a generic conversational infrastructure product. The AI Analytics Platform remains a governed analytical infrastructure product. Neither product needs to own what the other provides, and both can be deployed independently or together.

Consumption modes

The AI Analytics Platform is designed for three modes of consumption, which may be used simultaneously by the same host application:

Mode	Consumer	When to use
Direct API consumer	A custom application calling `POST /v1/mcp` directly — rendering the returned JSON result sets and Vega-Lite specs using their own UI stack	Dedicated analytics views, dashboards, analytical workbooks built by the host team — where the application owns the rendering layer entirely
Conversational backend	AI Chat Platform, registered via `mcpServers` config	Conversational analytics — where the user asks quantitative questions in natural language; the AI Chat Platform routes tool calls to the Analytics Platform's MCP endpoint
Agentic consumer	Autonomous AI agents, scheduled pipelines, event-triggered workflows	Any AI orchestrator that needs to query large datasets, monitor metrics, generate governed reports, or trigger analytical workflows without a human in the loop — all via the same MCP Capability Layer

All three modes route through the same governance pipeline. A portfolio manager drilling through a chart in a standalone view, asking the same question conversationally, and an overnight monitoring agent checking for limit breaches all receive results with identical metric definitions, role enforcement, and lineage provenance. There is no privileged path and no governance bypass for any consumer class.

Combined platform architecture

The following diagram shows all three consumption modes operating against the same Analytics Platform backend:

No consumer — AI Chat Platform, `<ai-analytics>` component, or agentic agent — has a path to execution engines, physical schemas, or raw SQL. Every analytical request routes through the MCP Capability Layer and the full governance pipeline. The Analytical Lineage Store records every invocation, regardless of which consumer initiated it.

MCP registration

To connect the Analytics Platform to the AI Chat Platform, a host application adds the following entry to the `mcpServers` section of its AI Chat Platform application config:

```
{
  "id":          "analytics-platform",
  "name":        "Analytics Platform",
  "description": "Governed analytical query engine. Resolves quantitative questions against
registered business metrics via the Semantic Metrics Registry. Use for portfolio performance,
risk decomposition, performance attribution, issuer concentration, regulatory metrics, and any
question requiring large-scale data analysis. Always specify metric IDs — do not attempt to
construct SQL or use unregistered identifiers. Call list_metrics first if unsure which metrics
are available.",
  "endpoint":    "https://api.analytics-platform.io/v1/mcp",
  "authType":    "bearer",
  "accessTier":  "always-on",
  "roles":       []
}
```

The `description` field is injected verbatim into the AI Chat Platform's system prompt and is the primary signal the AI model uses to decide when to route a question to the Analytics Platform versus other registered MCP servers. The description above is the recommended starting point; host teams should customise it to reflect their specific metric domains.

Setting `accessTier: "always-on"` ensures the Analytics Platform's capabilities are available in every session without user action — appropriate because most quantitative questions in a governed financial application require the Analytics Platform by default.

End-to-end conversational analytics flow

When a user asks a quantitative question in the AI Chat Platform, the complete flow from natural language to rendered result is:

```
User (in AI Chat Platform):
  "Show me the tracking error for each equity portfolio over the last quarter,
   and flag any that are above their mandate limit."

  ↓  AI Chat Platform routes to AI model with Analytics Platform capabilities in context

AI model:
  Resolves question → analyses_metric tool call
  {
    "metrics":    ["tracking_error", "tracking_error_limit"],
    "dimensions": ["portfolio"],
    "time_period": "quarter_to_date",
    "filters": [{ "dimension": "asset_class", "operator": "eq", "value": "EQUITY" }],
    "order_by": "tracking_error DESC"
  }

  ↓  AI Chat Platform forwards tool call to Analytics Platform MCP endpoint
     with the user's JWT

AI Analytics Platform — governance pipeline:
  1. Input schema validation
  2. Capability availability check (feature flags + user role)
  3. SMR resolution: verify tracking_error, tracking_error_limit, portfolio dimension
  4. Role-Aware Projection: filter to portfolios the user is entitled to see
  5. DSL compilation → Logical Query Plan
  6. Semantic Execution Governance: cost estimate check, compliance classification
  7. Federated Query Planner: route to registered execution engine(s)
  8. Result assembly + lineage record written

  ↓  Returns governed MCP tool response to AI Chat Platform:
     {
       "result_id":    "res_...",
       "display_spec": {
         "type":    "vega-lite",
         "$schema": "https://vega.github.io/schema/vega-lite/v5.json",
         "mark":    "bar",
         "data":    { "values": [ ... ] },
         "encoding": { ... }
       },
       "narrative": null   ← narrative generated in step below
     }

AI Chat Platform:
  - Renders result as data table + Vega-Lite bar chart (from the returned spec) in
conversation thread
  - Displays tool call disclosure card (collapsible): inputs, result_id, latency
  - AI model generates governed narrative:
    "Three portfolios are above their tracking error limit this quarter:
     Global Equity (4.2% vs 3.5% limit), EM Growth (5.1% vs 4.0% limit), ..."
    (values anchored exclusively to the execution result — hallucination prohibited)

User:
  "Drill into the Global Equity portfolio — break down what's driving the tracking error."

  ↓  AI model resolves to risk_breakdown tool call
     { "portfolio_id": "global-equity", "risk_metric": "tracking_error",
       "attribution_by": "factor", "as_of_date": "2026-03-31" }

  ↓  Full governance pipeline repeats; factor attribution result returned and rendered
```

At no point in this flow does the AI model construct SQL, access a physical schema, or produce a number not present in the Analytics Platform's execution result. Every quantitative claim in the conversation is backed by a lineage record in the Analytics Platform's Analytical Lineage Store, reachable via the `result_id` in the tool call disclosure card.

What the integration enables

Capability	Provided by
Large-scale, multi-engine data analytics from a conversation	Analytics Platform FQP routes governed plans to Snowflake, Databricks, Trino, or any registered engine — irrespective of result set size
Role-aware results without any configuration in the chat layer	Analytics Platform's Role-Aware Projection Layer enforces entitlements before results leave the analytical backend; the AI Chat Platform surfaces only what the authenticated user is permitted to see
Governed metric vocabulary, not ad hoc SQL	The AI model calls named, registered capabilities (`analyse_metric` , `risk_breakdown` , etc.) — the Analytics Platform's SMR ensures metric definitions are consistent, versioned, and owned
Deterministic chart specification	The Analytics Platform's Visualisation Ontology selects chart types based on the result schema and metric semantics and returns a Vega-Lite spec — the AI Chat Platform renders that spec, not a chart chosen ad hoc by the LLM
Governed narrative synthesis	The Narrative Synthesis Engine produces prose explanations anchored exclusively to execution result values; metric hallucination is architecturally prohibited
Full analytical lineage from every conversation turn	Every tool call disclosure card in the AI Chat Platform carries a `result_id` that maps to a complete lineage record — from natural-language intent through SMR resolution, projection, plan compilation, execution, and result delivery
Complex institutional workflows as simple conversational queries	Performance attribution, issuer concentration, risk decomposition, regulatory compliance checks — exposed as simple tool calls the AI model can compose in response to natural-language questions
Drilldown continuity across turns	The `drilldown` capability accepts a `result_id` from a prior turn, preserving parent-level filters and governance context as the user deepens their analysis across multiple conversation turns

---

Routing alongside other MCP servers (AI Chat Platform)

Host applications that deploy both platforms will typically register the Analytics Platform alongside their own operational MCP servers. The AI model routes between them based on the `description` fields in the `mcpServers` config:

Server	Handles	Example queries
Analytics Platform MCP	Large-scale, governed metric computation; federated query; attribution and decomposition	*"What is the tracking error for my equity portfolios?"*, *"Break down the risk drivers for Global Equity"*
Host application MCP	Operational data, workflow actions, entity lookups, real-time state (balances, positions, alerts, approval queues)	*"Show me the latest valuation for fund XYZ"*, *"What trades are pending settlement today?"*
Web Search MCP	Current market news, regulatory announcements, external reference data	*"What did the Fed announce yesterday?"*, *"What is the current SOFR rate?"*

For quantitative analytical questions, the AI model routes to the Analytics Platform. For operational lookups and real-time state, it routes to the host application's own servers. The `description` field on each server is the primary routing signal — the suggested `description` text above is optimised for correct routing in a financial services deployment.

Agentic consumers

Beyond the AI Chat Platform, the Analytics Platform's MCP Capability Layer is designed to serve any MCP-compatible AI orchestrator. Anticipated agentic consumers include:

Agent type	Example use case
Scheduled analytical agents	Nightly portfolio risk summary generated against the SMR and delivered as a governed narrative; daily regulatory metric check run before market open
Event-triggered monitors	An agent that runs `risk_breakdown` automatically when a tracking error threshold is breached, producing a governed analysis for the investment committee
Report-generation pipelines	Automated investment committee pack generation — `performance_attribution`, `compare_portfolios`, and `regulatory_metric` composed into a governed narrative document; Vega-Lite specs converted to static images via the `vite2img` service for PDF embedding
Compliance review agents	Periodic mandate compliance checks using `issuer_concentration` and `regulatory_metric`, with results written to an audit log
Research augmentation agents	Agents that combine web search (current news) with governed `analyse_metric` results to produce investment research anchored to verified portfolio data

All agentic consumers must present a valid JWT containing role claims. The Analytics Platform's Role-Aware Projection Layer enforces the same entitlement model as for human users — an agent operating with a portfolio manager's JWT receives exactly the same metric visibility and row-level security that the portfolio manager receives interactively. There is no elevated-privilege path for agents.

01 — Host Application Configuration

Every tenant on the AI Analytics Platform is defined by a **JSON application config** provided by the host application at registration time. The config is the single source of truth for how the analytics layer behaves within that application — its semantic domain, execution engines, entitlement model, metric registry scope, visualisation preferences, and feature flags.

Config delivery

Configs are registered and updated via the **Platform Admin API** (authenticated by a platform API key scoped to the tenant). A web-based **Config Editor UI** provides a visual wrapper over the same API for non-technical Application Admins. Both target the same config document.

Config takes effect on the **next new analytical session** after the update is applied. Active sessions are not interrupted by config changes.

Config schema

```
{
  "$schema": "https://analytics-platform.io/config/v1/schema.json",
  "version": "1.0.0",

  "identity":          { ... },
  "branding":          { ... },
  "scope":             { ... },
  "executionEngines":  [ ... ],
  "metricRegistry":    { ... },
  "entitlements":      { ... },
  "visualisation":     { ... },
  "drilldown":         { ... },
  "governance":        { ... },
  "models":            { ... },
  "conversations":     { ... },
  "features":          { ... }
}
```

`identity`

```
{
  "identity": {
    "tenantId":              "acme-wealth",
    "applicationName":       "Acme Wealth Management Platform",
    "analyticsName":         "Meridian Analytics",
    "analyticsDescription":  "Governed portfolio analytics for Acme Wealth Management.",
    "analyticalDomain":      "wealth_management",
    "regulatoryJurisdiction": "UK",
    "supportUrl":            "https://help.acme.com/meridian",
    "privacyPolicyUrl":      "https://acme.com/privacy"
  }
}
```

Field	Required	Description
`tenantId`	Yes	Unique slug identifier for this tenant on the platform
`applicationName`	Yes	Name of the host application — shown in platform admin UI
`analyticsName`	Yes	The name end users see for the analytics layer (e.g. "Meridian Analytics", "Apex Insights")
`analyticsDescription`	Yes	One-sentence description shown in the onboarding welcome state
`analyticalDomain`	Yes	Domain identifier used to seed pre-built metric registry templates. Accepted values: `wealth_management`, `banking`, `investment_management`, `institutional_analytics`, `risk_management`, `custom`
`regulatoryJurisdiction`	No	Primary regulatory jurisdiction — affects default compliance classification rules. Accepted: `UK`, `EU`, `US`, `SG`, `AU`, `HK`, or a comma-separated list for multi-jurisdictional deployments.
`supportUrl`	No	URL linked from error states for end-user support
`privacyPolicyUrl`	No	URL displayed in data access consent UI

`branding`

```
{
  "branding": {
    "colorPrimary":        "#003366",
    "colorSurface":        "#FFFFFF",
    "colorSurfaceVariant": "#F2F5F8",
    "colorOnSurface":      "#111827",
    "colorAccent":         "#00875A",
    "fontFamily":          "Inter, system-ui, sans-serif",
    "borderRadius":        "6px",
    "logoUrl":             "https://cdn.acme.com/logo.svg"
  }
}
```

Design tokens applied to the `<ai-analytics>` web component. Platform defaults apply for any token not provided.

`scope`

Defines the analytical assistant's domain, what queries it should serve, and how it handles out-of-scope requests.

```
{
  "scope": {
    "systemPrompt":          "You are Meridian, the governed analytics assistant for Acme
Wealth Management. You help portfolio managers, risk officers, and relationship managers
access portfolio analytics, risk metrics, and performance data using governed semantic
definitions...",
    "analyticalScope":       "Portfolio analytics, risk metrics, performance attribution, and
regulatory reporting within Acme Wealth Management",
    "outOfScopeRedirect":    "I'm scoped to governed portfolio and risk analytics for Acme
Wealth Management. For general market research, please use the research portal.",
    "language":              "en",
    "requiresIntentConfirmation": false
  }
}
```

Field	Required	Description
`systemPrompt`	Yes	Base system prompt for the analytical assistant. The platform appends metric registry context, entitlement summaries, and governance instructions.
`analyticalScope`	Yes	Plain-language description of analytical scope — injected into user-facing scope descriptions and out-of-scope redirect messages.
`outOfScopeRedirect`	No	Custom message for out-of-scope queries. Platform uses a generic fallback if not set.
`language`	No	BCP 47 language tag (default: `en`).
`requiresIntentConfirmation`	No	When `true`, the platform surfaces a structured intent confirmation card to the user before executing any analytical plan. Recommended for high-governance environments. Default: `false`.

`executionEngines`

Registers the analytical execution engines available within this tenant. The platform's Federated Query Planner routes Logical Query Plan fragments to registered engines based on capability declarations and data affinity.

```
{
  "executionEngines": [
    {
      "id":           "snowflake-primary",
      "name":         "Primary Data Warehouse",
      "type":         "snowflake",
      "endpoint":     "https://api.acme.com/analytics/engines/snowflake",
      "authType":     "bearer",
      "capabilities": ["aggregate", "filter", "join", "window", "timeseries"],
      "dataAffinity": ["portfolio", "positions", "transactions", "benchmarks"],
      "priority":     1,
      "costTier":     "standard",
      "enabled":      true
    },
    {
      "id":           "dbt-semantic",
      "name":         "dbt Semantic Layer",
      "type":         "dbt_semantic_layer",
      "endpoint":     "https://api.acme.com/analytics/engines/dbt",
      "authType":     "api-key",
      "capabilities": ["metric", "dimension", "filter"],
      "dataAffinity": ["risk_metrics", "performance_attribution"],
      "priority":     2,
      "costTier":     "low",
      "enabled":      true
    },
    {
      "id":           "cube-olap",
      "name":         "OLAP Cache Layer",
      "type":         "cube",
      "endpoint":     "https://api.acme.com/analytics/engines/cube",
      "authType":     "bearer",
      "capabilities": ["aggregate", "filter", "pre-aggregated"],
      "dataAffinity": ["summary_metrics", "dashboard_kpis"],
      "priority":     3,
      "costTier":     "minimal",
      "enabled":      true
    }
  ]
}
```

Field	Required	Description
`id`	Yes	Unique identifier within the tenant
`name`	Yes	Display name shown in lineage records and admin UI
`type`	Yes	Engine type — determines the physical query dialect the FQP generates. Accepted: `snowflake`, `bigquery`, `databricks`, `trino`, `dbt_semantic_layer`, `cube`, `redshift`, `postgres`, `custom`
`endpoint`	Yes	Platform-facing HTTPS endpoint that accepts LQP fragments and returns result sets per the platform's engine adapter protocol
`authType`	Yes	`bearer` (user JWT forwarded), `api-key` (platform holds key per tenant), `service-account`

Field	Required	Description
`capabilities`	Yes	Array of query capabilities this engine supports. Used by the FQP for sub-plan routing. Accepted: `aggregate`, `filter`, `join`, `window`, `timeseries`, `metric`, `dimension`, `pre-aggregated`
`dataAffinity`	Yes	Logical data domains this engine is the preferred source for. Used by the FQP to route sub-plans. Must reference domains declared in the SMR.
`priority`	No	Routing priority when multiple engines can serve a sub-plan (lower = higher priority). Default: `10`.
`costTier`	No	Execution cost tier — used by the governance circuit breaker. Accepted: `minimal`, `low`, `standard`, `high`, `unrestricted`. Default: `standard`.
`enabled`	No	Whether this engine is available for routing. Default: `true`.

`metricRegistry`

Configures the Semantic Metrics Registry (SMR) for this tenant.

```
{
  "metricRegistry": {
    "seedTemplate":        "wealth_management",
    "customMetricsUrl":    "https://api.acme.com/analytics/metrics/registry",
    "refreshIntervalMins": 15,
    "versionControl":      true,
    "approvalRequired":    true,
    "ownershipRequired":   true,
    "maxMetrics":          500,
    "defaultAggregation":  "sum",
    "fiscalYearStartMonth": 1
  }
}
```

Field	Required	Description
`seedTemplate`	No	Pre-built metric definitions to seed the registry from. Accepted: `wealth_management`, `banking`, `investment_management`, `risk_management`, `regulatory`. Multiple can be specified as an array.
`customMetricsUrl`	No	Host-provided endpoint from which the platform fetches the tenant's custom metric definitions (YAML or JSON, per the SMR schema in 04-semantic-metrics-registry.md). Polled at `refreshIntervalMins`.
`refreshIntervalMins`	No	How often the platform re-fetches metric definitions from `customMetricsUrl`. Default: `60`.
`versionControl`	No	Whether metric definitions are version-controlled in the platform's registry store. Default: `true`. Highly recommended — version history enables lineage reconstruction.

Field	Required	Description
`approvalRequired`	No	Whether new or modified metric definitions require approval before becoming resolvable. Default: `true`.
`ownershipRequired`	No	Whether every metric must have an assigned owner before becoming resolvable. Default: `true`.
`maxMetrics`	No	Maximum number of active metrics in the tenant registry. Default: `500`.
`defaultAggregation`	No	Default aggregation rule applied when a metric definition does not specify one. Default: `sum`.
`fiscalYearStartMonth`	No	Integer (1–12) for fiscal year calculations. Default: `1` (January).

`entitlements`

Configures the Role-Aware Projection Layer for this tenant.

```
{
  "entitlements": {
    "roleClaimField":     "analytics_roles",
    "defaultDenyAll":     true,
    "rowSecurityMode":    "predicate_injection",
    "columnMaskingMode":  "null_replacement",
    "roles": [
      {
        "id":              "portfolio_manager",
        "label":           "Portfolio Manager",
        "metricAccess":    ["aum", "portfolio_return", "benchmark_return", "tracking_error",
"sharpe_ratio"],
        "dimensionAccess": ["portfolio", "asset_class", "geography", "currency", "date"],
        "rowPredicates":   ["portfolio_id IN ({{user.managed_portfolios}})"],
        "columnMasks":     []
      },
      {
        "id":              "risk_officer",
        "label":           "Risk Officer",
        "metricAccess":    ["var_95", "var_99", "cvar", "beta", "duration",
"credit_spread_dv01", "issuer_concentration"],
        "dimensionAccess": ["portfolio", "asset_class", "issuer", "rating", "date"],
        "rowPredicates":   [],
        "columnMasks":     []
      },
      {
        "id":              "compliance_analyst",
        "label":           "Compliance Analyst",
        "metricAccess":    ["regulatory_capital", "lcr", "nsfr", "leverage_ratio"],
        "dimensionAccess": ["entity", "regulatory_classification", "jurisdiction", "date"],
        "rowPredicates":   [],
        "columnMasks":     ["client_name", "account_number"]
      }
    ]
  }
}
```

Field	Required	Description
`roleClaimField`	Yes	The JWT claim key containing the user's role array
`defaultDenyAll`	No	When `true`, users with no matching role see no metrics. When `false`, users with no matching role receive a public-access metric set. Default: `true`.
`rowSecurityMode`	No	How row-level security is applied. `predicate_injection` — SQL predicates injected into physical queries; `pre-filter` — result sets filtered after FQP assembly. Default: `predicate_injection`.
`columnMaskingMode`	No	How masked columns are handled. `null_replacement` — masked values replaced with null; `redacted_label` — replaced with `[REDACTED]`; `excluded` — column omitted from result. Default: `null_replacement`.
`roles`	Yes	Array of role definitions. Each role maps to a metric access list, dimension access list, row predicates, and column masks.

Role definition fields

Field	Description
`id`	Unique role identifier — must match values in user JWTs
`label`	Display name for admin UI and lineage records
`metricAccess`	Array of metric IDs (from the SMR) this role may query. Wildcard `["*"]` grants access to all registered metrics.
`dimensionAccess`	Array of dimension IDs this role may slice by. Wildcard `["*"]` grants all dimensions.
`rowPredicates`	SQL-like predicate fragments injected at execution time. Use `{{user.claim_name}}` to reference JWT claim values.
`columnMasks`	Array of physical column names to mask for this role. Applied by the FQP before result assembly.

`visualisation`

Configures the Visualisation Ontology and rendering preferences for this tenant.

```
{
  "visualisation": {
    "defaultChartLibrary": "vega-lite",
    "allowedChartTypes":   ["bar", "line", "area", "scatter", "heatmap", "treemap",
"waterfall", "gauge", "table", "sparkline"],
    "colorPalette":        "categorical_financial",
    "numberFormat": {
      "currency":   "GBP",
      "locale":     "en-GB",
      "decimals":   2,
      "largeNumbers": "abbreviated"
    },
    "thresholds": {
      "maxDataPoints": 10000,
      "maxSeriesPerChart": 20
    },
    "customRenderers": [
      {
        "id":          "risk-heatmap",
        "trigger":     "risk-heatmap",
        "name":        "Risk Heatmap",
        "moduleUrl":   "https://cdn.acme.com/analytics-renderers/risk-heatmap.js"
      }
    ]
  }
}
```

Field	Description
`defaultChartLibrary`	Rendering library for standard chart types. Accepted: `vega-lite`, `plotly`, `echarts`. Default: `vega-lite`.
`allowedChartTypes`	Chart types the Visualisation Ontology may select for this tenant. Restricting this set enforces consistency across analytical outputs.
`colorPalette`	Named colour scheme for chart series. Platform-provided options: `categorical_financial`, `sequential_blue`, `diverging_red_green`, `monochrome`.
`numberFormat`	Default number formatting applied to all metric values in visualisations and narrative synthesis.
`thresholds.maxDataPoints`	Maximum data points returned to the visualisation layer. Queries exceeding this threshold trigger the governance circuit breaker.
`thresholds.maxSeriesPerChart`	Maximum chart series. Queries resolving more series trigger automatic aggregation by the FQP.
`customRenderers`	Host-registered domain-specific visualisation renderers, loaded as ES modules.

`drilldown`

Configures the governed drilldown behaviour — how users and AI agents traverse analytical hierarchies.

```
{
  "drilldown": {
    "enabled":           true,
    "maxDepth":          4,
    "allowedHierarchies": ["asset_class_hierarchy", "geography_hierarchy", "time_hierarchy",
"organisational_hierarchy"],
    "requireConfirmation": false,
    "preserveFilters":   true
  }
}
```

Field	Description
`enabled`	Whether governed drilldown is available in this tenant. Default: `true`.
`maxDepth`	Maximum levels of hierarchy traversal per analytical session. Prevents runaway recursive queries. Default: `4`.
`allowedHierarchies`	Hierarchy IDs (defined in the SMR) that drilldown may traverse. Hierarchies not listed here cannot be traversed interactively.
`requireConfirmation`	When `true`, the platform presents a drilldown confirmation step before traversing. Default: `false`.
`preserveFilters`	Whether parent-level filters carry forward to child levels during drilldown. Default: `true`.

`governance`

Configures the Semantic Execution Governance layer for this tenant.

```
{
  "governance": {
    "maxQueryCostUnits":          1000,
    "maxConcurrentQueries":       5,
    "queryTimeoutSeconds":        60,
    "classificationGating":       true,
    "blockedClassifications":     ["TOP_SECRET", "RESTRICTED"],
    "requireLineageForExport":    true,
    "auditAllQueries":            true,
    "complianceMode":             "mifid2"
  }
}
```

Field	Description
`maxQueryCostUnits`	Maximum estimated cost units for a single analytical query. Queries exceeding this limit are blocked and the user is prompted to narrow scope.
`maxConcurrentQueries`	Per-user concurrency limit. Default: `5`.
`queryTimeoutSeconds`	Maximum execution time before the FQP cancels the query. Default: `60`.

Field	Description
`classificationGating`	When `true`, data classification metadata is evaluated before execution — queries touching data at or above `blockedClassifications` are blocked. Default: `true`.
`blockedClassifications`	Data classification labels that block query execution. Sourced from the host's data classification taxonomy.
`requireLineageForExport`	When `true`, analytical results cannot be exported until the lineage record is fully written. Default: `true`.
`auditAllQueries`	When `true`, every query — including those that fail governance checks — is written to the audit trail. Default: `true`.
`complianceMode`	Named compliance profile that pre-configures governance defaults. Accepted: `mifid2`, `basel3`, `aifmd`, `sec_reg_bi`, `mas_faa`, `custom`.

`models`

```
{
  "models": {
    "intentResolutionModel": "standard",
    "narrativeSynthesisModel": "standard",
    "allowedModels":          ["standard", "powerful"],
    "provider":               "anthropic"
  }
}
```

Field	Description
`intentResolutionModel`	Model tier used for semantic intent resolution (mapping natural language to analytical intent). Default: `standard`.
`narrativeSynthesisModel`	Model tier used for narrative synthesis. May differ from intent resolution — more complex narratives may warrant the `powerful` tier. Default: `standard`.
`allowedModels`	Model tiers available for this tenant.
`provider`	AI provider identifier.

`features`

```
{
  "features": {
    "naturalLanguageQuery":    true,
    "governedDrilldown":       true,
    "narrativeSynthesis":      true,
    "exportResults":           true,
    "lineageInspector":        true,
    "intentConfirmation":      false,
    "benchmarkComparison":     true,
    "regulatoryReporting":     false,
    "starterQuestions": [
      "What is the portfolio return versus benchmark for the current quarter?",
      "Show me issuer concentration risk across all portfolios.",
      "Which portfolios have VaR exceeding their limit today?"
    ]
  }
}
```

Flag	Default	Description
`naturalLanguageQuery`	`true`	Enable natural language intent resolution
`governedDrilldown`	`true`	Enable hierarchy traversal
`narrativeSynthesis`	`true`	Enable LLM-generated prose anchored to results
`exportResults`	`true`	Enable result export (CSV, JSON, PDF)
`lineageInspector`	`true`	Enable the lineage trail inspector UI
`intentConfirmation`	`false`	Require user confirmation before plan execution
`benchmarkComparison`	`true`	Enable benchmark dimension access
`regulatoryReporting`	`false`	Enable regulatory metric domain (requires appropriate role)
`starterQuestions`	`[]`	Suggested analytical questions shown on the onboarding screen (up to 3)

Config validation

Submitting a config via the Admin API runs synchronous validation before applying:

Check	Description
Schema conformance	All required fields present; field types match schema
Execution engine reachability	Engine endpoints respond to a health check
Role claim field presence	`entitlements.roleClaimField` resolvable from test JWT

Check	Description
Metric access references	All metric IDs in role `metricAccess` arrays must exist in the seeded SMR or be declared as custom metrics
Hierarchy references in drilldown	All `allowedHierarchies` IDs must be defined in the SMR
Governance mode compatibility	`complianceMode` values compatible with `regulatoryJurisdiction`
Custom renderer reachability	Each `moduleUrl` must respond to a HEAD request
System prompt token count	Must be under 3,000 tokens

Validation errors return a structured response with field-level detail. Config is not applied until all checks pass.

02 — Personas and User Journeys

The AI Analytics Platform is deployed by host applications across regulated financial services contexts. This document describes personas at two levels: **platform-level archetypes** (roles that exist across all deployments) and **illustrative host application journeys** (examples showing how different financial services contexts use the platform).

Platform-level personas

Persona	Role	Primary need
Analytical End User	Authenticated user of the host application using the analytics layer as their primary data access interface	Ask governed analytical questions and receive reliable, role-appropriate results without knowledge of underlying data structures or metric definitions
Power Analyst	Experienced user who composes multi-dimensional analytical requests and navigates drilldown hierarchies	Multi-dimensional analytical exploration, governed drilldown, lineage inspection, result export for downstream use
Application Admin	Privileged user within the tenant responsible for the SMR, entitlement policies, and governance configuration	Manage metric definitions, approve registry changes, maintain entitlement policies, review governance audit trail
Metric Owner	Subject-matter expert assigned ownership of one or more metric definitions in the SMR	Review proposed changes to owned metrics, approve aggregation rule changes, maintain metric documentation
Host Developer	Engineer at the host application team responsible for config and engine integration	Register execution engines, maintain the application config, integrate entitlement model
Platform Admin	AI Analytics Platform team member with cross-tenant visibility	Platform health, tenant onboarding, infrastructure, governance audit review

Application Admin design note

The Application Admin role is the platform's equivalent of a Chief Data Officer or Head of Analytics Data Governance within the tenant. They are responsible for the integrity of the SMR — what can be queried, how metrics are defined, and who can access what. Key responsibilities:

- Managing the Semantic Metrics Registry — approving new metric definitions, deprecating obsolete ones, resolving definition conflicts
- Maintaining entitlement policies — role-to-metric-access mappings, row predicates, column masks
- Reviewing the governance audit trail — flagging unusual query patterns or policy violations
- Configuring execution engine routing priorities and cost limits
- Approving Application Context for the analytical assistant

This role must exist in every tenant before go-live.

Illustrative host application journeys

Journey A — Wealth management: portfolio morning briefing

Host application type: Wealth management platform
Persona: Portfolio Manager (Analytical End User)
Setting: Desktop, first thing in the morning

A portfolio manager opens the analytics layer and types: *"Show me portfolio returns versus benchmark across all my portfolios for the current quarter, sorted by tracking error."*

The platform resolves the intent against the SMR, identifying metrics `portfolio_return`, `benchmark_return`, `tracking_error` and the `portfolio` and `date` dimensions. The Role-Aware Projection Layer injects a row predicate scoped to this manager's assigned portfolios from their JWT claims.

The platform produces: - A bar chart (Visualisation Ontology selects: multi-series bar, metric-to-benchmark comparison pattern) showing quarterly return vs. benchmark by portfolio - A ranked data table by tracking error - A narrative synthesis: *"Across 14 portfolios, 9 outperformed their benchmark this quarter. The Global Equity Opportunities portfolio has the highest tracking error at 3.2%, driven primarily by significant overweight positions in Technology. Two portfolios — UK Core Income and Strategic Balanced — are within their tracking error budget."*

The manager clicks the Technology bar segment. A governed drilldown traverses the `asset_class_hierarchy` to the sector level, showing sector-level attribution.

Features exercised: Natural language intent resolution, role-aware projection, multi-metric query, Visualisation Ontology chart selection, narrative synthesis, governed drilldown, lineage trail.

Journey B — Risk management: VaR limit breach investigation

Host application type: Investment risk management system
Persona: Risk Officer (Power Analyst)
Setting: Desktop, intraday monitoring

A risk officer receives an alert. They open the analytics layer and ask: *"Which portfolios are breaching their VaR 95 limit today, and what is the dominant risk factor contribution for each?"*

The platform resolves `var_95`, `var_limit`, `risk_factor_contribution` metrics and the `portfolio` and `date` dimensions. The intent confirmation card is not shown (disabled for this tenant). The FQP routes the VaR metrics to the risk engine and the portfolio data to the primary warehouse.

The result is: - A heatmap (Visualisation Ontology: metric-vs-threshold pattern → heatmap contract) showing breach severity by portfolio - A drilldown-ready table of limit breach amounts - Narrative

synthesis: *"3 portfolios are breaching their VaR 95 limit today. Emerging Markets High Yield has the most severe breach at 142% of limit. The dominant risk factor for all three portfolios is credit spread widening in BBB-rated corporate bonds, accounting for 64–71% of the excess VaR."*

The risk officer opens the lineage inspector. They can see exactly which execution engines were called, what predicates were applied, and which metric definitions from the SMR were used.

Features exercised: Multi-engine federation, VaR metrics, risk factor attribution, heatmap rendering, narrative synthesis, lineage inspector.

Journey C — Compliance: regulatory reporting preparation

Host application type: Banking compliance management platform
Persona: Compliance Analyst
Setting: Desktop, end-of-quarter regulatory reporting cycle

A compliance analyst needs to prepare the LCR (Liquidity Coverage Ratio) and NSFR (Net Stable Funding Ratio) summary for submission. They ask: *"Show me the current LCR and NSFR for all regulated entities, with a 30-day trend."*

The platform identifies that the `regulatory_reporting` feature flag is enabled and the user's role includes `compliance_analyst`. The Role-Aware Projection Layer applies column masks to client name and account number fields per the compliance role definition. The governance layer validates that the data classification of LCR and NSFR data is within the user's permitted classification level.

The result is: - A line chart showing 30-day trend of LCR and NSFR per entity - A summary table of current ratios vs. regulatory minima - An export-ready data table with the complete lineage record attached (required by `requireLineageForExport: true` in the governance config)

Features exercised: Regulatory metric domain, role-aware column masking, classification gating, lineage-gated export, compliance mode governance.

Journey D — Institutional analytics: issuer concentration review

Host application type: Institutional asset management platform
Persona: Power Analyst
Setting: Desktop, pre-investment committee

An analyst prepares for the investment committee meeting. They ask: *"Show me issuer concentration risk across all equity portfolios — highlight any issuer where the aggregate position exceeds 5% of total AUM."*

The platform resolves `issuer_concentration`, `aum`, `aggregate_position_pct` metrics and slices by `issuer` and `portfolio` dimensions. The Visualisation Ontology selects a treemap (issuer concentration pattern → treemap contract) with threshold highlighting.

The analyst then uses the drilldown to navigate from the issuer level to individual security positions within the highest-concentration issuer.

They export the result as PDF — the lineage record is automatically attached. The export is flagged in the audit trail as a pre-committee analytical artefact.

Features exercised: Issuer concentration metrics, treemap rendering, drilldown (issuer → security), PDF export with lineage, audit trail artefact flagging.

Journey E — Application Admin: metric definition governance

Host application type: Any financial services platform
Persona: Application Admin
Setting: Platform Admin UI

A metric owner has proposed a new metric: `modified_duration_weighted` — a portfolio-level modified duration weighted by market value. The Application Admin reviews the proposal in the SMR management UI:

- Metric label: Modified Duration (Market Value Weighted)
- Formula: `SUM(position_market_value × security_modified_duration) / SUM(position_market_value)`
- Aggregation: `value_weighted_average`
- Dimensions: `portfolio`, `asset_class`, `date`
- Data lineage: sourced from `positions` domain, `risk_metrics` sub-domain
- Owner: Head of Fixed Income Risk
- Access roles: `risk_officer`, `portfolio_manager`

The Application Admin reviews, notes that the formula is consistent with the existing `duration` metric's definition style, and approves. The metric is now resolvable in the SMR from the next refresh cycle.

Features exercised: SMR metric approval workflow, formula review, ownership assignment, role access policy, metric activation.

Persona × Feature Matrix

Feature	End User	Power Analyst	Compliance Analyst	App Admin	Metric Owner	Host Developer
Natural language query	✓	✓	✓	✓		
Metric resolution from SMR	✓	✓	✓	✓		
Role-aware results	✓	✓	✓	✓		
Governed drilldown		✓	✓	✓		
Lineage inspector		✓	✓	✓	✓	

Feature	End User	Power Analyst	Compliance Analyst	App Admin	Metric Owner	Host Developer
Narrative synthesis	✓	✓	✓	✓		
Result export	✓	✓	✓	✓		
SMR browsing (read-only)		✓	✓	✓	✓	
SMR metric proposal				✓	✓	
SMR metric approval				✓		
Entitlement policy management				✓		
Engine registration						✓
Governance audit trail			✓	✓		✓
Config management				✓		✓
Regulatory metric domain			✓	✓		

03 — Design Principles

These nine principles govern all design decisions for the AI Analytics Platform. Where a proposed feature conflicts with a principle, the principle takes precedence. Deviations require an explicit decision record.

P1 — Semantic abstraction over physical exposure

The platform never exposes physical storage schemas, table names, column names, or execution engine internals to AI models or end users. All AI interaction is mediated through the Semantic Metrics Registry. If a business concept is not registered in the SMR, it is not queryable. There is no escape hatch to raw SQL.

Consequences: - The Analytics DSL compiler operates exclusively on SMR-registered identifiers. Any attempt to introduce physical schema references is rejected at the compiler boundary. - Physical query generation is entirely the responsibility of the Federated Query Planner — the LLM has no access to or knowledge of physical schemas. - The system prompt injected into the AI model contains metric names, dimension names, and analytical concepts drawn from the SMR — not table schemas, column names, or JOIN paths. - Schema leakage — even partial — is a security and governance violation. The platform architecture makes this impossible by design, not by policy enforcement alone.

P2 — Governance before execution

Every analytical query passes through the governance pipeline before any physical execution begins. There is no fast path that bypasses the Role-Aware Projection Layer or the Semantic Execution Governance checks. Governance is not a post-processing filter — it is a pre-execution gate.

Consequences: - The execution sequence is invariant: Intent Resolution → SMR Resolution → Role-Aware Projection → DSL Compilation → Governance Validation → FQP → Execution. No step may be skipped. - Queries that fail governance checks are blocked entirely — they do not partially execute. Partial results from a governance-blocked query are never returned. - Governance decisions are logged before the query reaches an execution engine, so the audit trail reflects the governance state at decision time, not at result time. - Governance configuration changes (entitlement policies, classification gates, cost limits) take effect on the next query — there is no cache of pre-governance query plans.

P3 — Deterministic metric resolution

A given metric name must resolve to exactly one governed definition for a given tenant at a given point in time. There are no context-dependent interpretations of metric semantics. "Portfolio Return" means the same thing in every query, every dashboard, and every narrative — or it is a bug.

Consequences: - Metric definitions in the SMR are version-controlled. A metric name resolves to its approved definition at the time of query execution. Historical lineage records preserve the definition

version used. - Conflicting metric definitions are a configuration error — the platform rejects any SMR submission where a metric ID already exists with a different formula, unless the submission is an explicit version update. - The LLM may not infer metric definitions from context. If a metric is not in the SMR, the platform returns a resolution error — not an inferred or approximated definition. - User-visible metric labels, descriptions, and units are sourced from the SMR definition — the LLM does not paraphrase or reinterpret them.

P4 — Complete analytical lineage

Every analytical result has a complete, queryable lineage chain: the user's natural language intent, the resolved analytical intent, the DSL expression, the Logical Query Plan, the physical engine sub-plans, the execution engine responses, the assembled result set, and the applied governance decisions. This lineage is a first-class artefact.

Consequences: - Lineage records are written atomically with the result — a result with no lineage record is a platform defect. - The lineage inspector UI is available to Power Analysts and above for every analytical result in their session. - Exported analytical results include a lineage reference that can be used to reconstruct the full execution chain. - Regulatory audit requests can be satisfied by querying the lineage store directly. - Lineage records are not mutable after writing. Corrections produce a new lineage record referencing the original.

P5 — Role-aware by default

The platform applies entitlements by default — a user sees exactly the metrics, dimensions, and data rows their role entitles them to, without having to request restrictions. There is no opt-in to data governance. An unauthenticated or unentitled query is blocked before any resolution occurs.

Consequences: - JWT validation and role claim extraction happen before any analytical processing begins. - `defaultDenyAll: true` is the platform-recommended setting — users with no matching role see nothing, not a default public view. - Row predicates are injected at the physical query level — not as a post-processing filter that could leak data via error messages. - If a user's role changes mid-session, the new entitlements apply to subsequent queries. In-session results from before the role change are not retrospectively restricted — but their lineage records reflect the entitlements in force at execution time.

P6 — Governed narrative, not free narrative

Narrative synthesis (LLM-generated prose describing analytical results) is anchored exclusively to the values present in the execution result. The LLM may not introduce metric values, comparisons, or interpretations that are not directly derivable from the result set. Hallucinated financial metrics are a regulatory and reputational risk.

Consequences: - The narrative synthesis prompt is constructed from the execution result — the LLM receives the structured result values, not a free-form query context. - A system-level constraint is injected

into the narrative synthesis prompt: *"You may only reference metric values, dates, and identifiers present in the provided result set. You may not introduce external figures, estimates, or comparisons from your training data."* - Narrative synthesis outputs are validated post-generation against the result set — any numeric value in the narrative that does not appear in the result is flagged and the narrative is regenerated. - Users can inspect the source result set underlying any narrative via the lineage inspector.

P7 — Deterministic visualisation

Chart type selection is governed by the Visualisation Ontology — a registered set of chart contracts that map result schemas and intent patterns to specific chart configurations. The LLM does not select chart types ad hoc. This ensures that the same analytical pattern always produces the same chart type across users, sessions, and time.

Consequences: - The Visualisation Ontology defines: for a given result schema (metric types, dimension cardinality, time axis presence) and intent pattern (comparison, trend, distribution, composition, relationship), which chart contract applies. - Chart contract parameters (axis assignments, colour encoding, tooltip definitions, drilldown anchors) are derived algorithmically from the result schema — not inferred by the LLM. - Custom chart types must be registered as host renderer modules before they can be selected by the ontology. - The LLM may express an intent that suggests a chart preference (e.g. *"show me a breakdown"*), but the Visualisation Ontology makes the final chart selection — the LLM suggestion is treated as an intent signal, not a direct rendering instruction.

P8 — Explainability at every layer

Users and compliance functions must be able to understand exactly what was queried, why, and with what results at every layer of the analytical stack. Opacity is unacceptable in regulated financial analytics.

Consequences: - The intent confirmation card (when enabled) shows the user the resolved analytical intent before execution — giving them the opportunity to correct misinterpretations before a query is run. - The lineage inspector exposes every step of the execution chain in a structured, human-readable format. - Execution governance decisions (blocks, cost warnings, classification gates) are explained to the user in plain language — not just "query blocked". - SMR metric definitions are accessible to any authenticated user (within their entitlement scope) — users can inspect exactly how any metric they see is calculated. - The narrative synthesis output can be expanded to show the source result values it was anchored to.

P9 — Host sovereignty within governance bounds

The host application has final authority over the analytical configuration — which metrics are registered, how entitlements are structured, which execution engines are used, and what governance thresholds apply. The platform enforces a set of non-negotiable governance minimums (lineage, role-awareness, semantic abstraction) but within those bounds, the host is in control.

Consequences: - Host applications configure their SMR, entitlement model, execution engines, and governance thresholds via the application config and Admin API. - Platform-managed governance (no raw SQL, mandatory lineage, role-aware projection, semantic abstraction) are non-overridable — they cannot be disabled by host config. - Hosts may raise governance thresholds (tighter cost limits, stricter classification gates) but may not lower them below platform minimums. - New platform-level governance defaults that would affect existing tenants require a migration path and advance notice.

Principle interactions

Principle	Most common tension	Resolution
P1 (semantic abstraction) vs query expressiveness	Users or AI agents want to express analytical logic not in the SMR	Add the concept to the SMR — it is a governance and configuration task, not a platform limitation
P2 (governance before execution) vs query latency	Governance pipeline adds latency	Governance checks are optimised for sub-100ms execution; entitlement projections are cached per session
P3 (deterministic resolution) vs metric evolution	Metric definitions change over time	SMR version-controls metric definitions; lineage records preserve definition version at query time
P4 (complete lineage) vs storage cost	Lineage records consume significant storage	Lineage record retention is tenant-configurable; compressed lineage format used for long-term storage
P6 (governed narrative) vs narrative quality	Strict anchoring may produce less fluent prose	Narrative quality improves with richer result sets; the constraint prevents hallucination at the cost of occasional prosaic output
P7 (deterministic visualisation) vs user chart preferences	Users may prefer a different chart type	Ontology includes an override mechanism for Power Analysts — overrides are logged in the lineage record
P8 (explainability) vs UX simplicity	Full lineage exposure may overwhelm casual users	Lineage inspector is progressive disclosure — collapsed by default, expandable by Power Analysts
P9 (host sovereignty) vs P2 (governance before execution)	Host wants to bypass governance for internal tools	Governance minimums are absolute — the platform does not provide a governance bypass mode

04 — Semantic Metrics Registry

Overview

The **Semantic Metrics Registry (SMR)** is the governing catalogue of every analytical concept resolvable on the platform. It is the single source of truth for what can be queried, how metrics are defined, what dimensions are available, how hierarchies are structured, who owns each definition, and what lineage metadata applies.

Nothing is queryable that is not registered in the SMR. This is an architectural constraint, not a policy. The Analytics DSL compiler rejects any metric identifier not present in the SMR for the active tenant.

Registry structure

The SMR is composed of five interconnected concept types:

Concept type	Description
Metric	A quantitative measure with a governed formula, aggregation rule, and unit
Dimension	A categorical or temporal attribute by which metrics can be sliced and filtered
Hierarchy	An ordered set of dimensions forming a navigable analytical hierarchy (for drilldown)
Measure Group	A named collection of related metrics grouped for analytical coherence
Domain	A logical grouping of metrics, dimensions, and hierarchies sharing a common business subject area

---

Metric definition schema

Every metric in the SMR conforms to the following schema:

```
metric:
  id:               "portfolio_return"
  version:          "2.1.0"
  label:            "Portfolio Return"
  description:      "Total return of a portfolio over the specified period, net of fees,
expressed as a percentage."
  formula:          "(end_market_value - start_market_value + cash_flows) /
start_market_value"
  unit:             "percentage"
  aggregation:
    default:        "value_weighted_average"
    allowed:        ["value_weighted_average", "equal_weighted_average", "sum"]
    granularity:    ["daily", "monthly", "quarterly", "annual", "since_inception"]
  dimensions:
    required:       ["portfolio", "date"]
    optional:       ["asset_class", "currency", "benchmark"]
  data:
    domain:         "portfolio"
    sub_domain:     "performance"
    source_tables:  ["fact_portfolio_daily", "dim_portfolio"]
    refresh_cadence: "daily"
    latency_sla:    "T+1"
  governance:
    owner:          "head_of_performance_analytics"
    steward:        "performance_analytics_team"
    classification: "INTERNAL"
    approved:       true
    approved_by:    "cdo_office"
    approved_at:    "2025-11-15T09:00:00Z"
    effective_from: "2025-11-15"
    deprecated:     false
  lineage:
    upstream_metrics: []
    upstream_sources: ["positions_service", "pricing_service", "cash_flow_service"]
    downstream_metrics: ["active_return", "information_ratio"]
  access:
    roles:          ["portfolio_manager", "risk_officer", "application_admin"]
    public:         false
  display:
    format:         "percentage"
    decimals:       2
    sign_convention: "positive_is_gain"
    benchmark_comparison: true
```

Metric schema field reference

Field	Required	Description
`id`	Yes	Unique metric identifier within the tenant. Lowercase, underscores. Used in DSL expressions and API calls.
`version`	Yes	Semantic version. Increment on formula changes (major), aggregation changes (minor), documentation changes (patch).
`label`	Yes	Human-readable metric name. Used in UI, narrative synthesis, and chart axis labels.

Field	Required	Description
`description`	Yes	Full prose definition. Must be unambiguous — this definition is injected into the AI model context.
`formula`	Yes	Business-logic formula expressed in the platform's formula language (see below). Does not reference physical table columns directly — references other SMR metrics or canonical data source identifiers.
`unit`	Yes	Value unit. Accepted: `percentage`, `currency`, `basis_points`, `ratio`, `count`, `years`, `days`, `custom`.
`aggregation.default`	Yes	Default aggregation rule when the metric is rolled up across a dimension.
`aggregation.allowed`	Yes	All permitted aggregation rules for this metric. Requests using non-allowed rules are rejected at DSL compilation.
`aggregation.granularity`	Yes	Time granularities at which this metric is calculable. Requests at unsupported granularities are rejected.
`dimensions.required`	Yes	Dimensions that must be present in any query using this metric. Missing required dimensions cause a DSL compilation error.
`dimensions.optional`	No	Dimensions that may optionally be applied.
`data.domain`	Yes	The logical data domain this metric belongs to. Must match a domain registered in the SMR.
`data.refresh_cadence`	Yes	How frequently the underlying data is updated. Displayed in the lineage inspector and narrative synthesis.
`governance.owner`	Yes	Identifier of the metric owner. Must be a registered owner in the platform.
`governance.classification`	Yes	Data classification level. Used by the governance classification gate.
`governance.approved`	Yes	Whether this metric has been approved and is resolvable. Unapproved metrics are not returned by SMR queries.
`lineage.upstream_metrics`	No	Other SMR metrics this metric is derived from. Used for lineage graph construction.
`lineage.downstream_metrics`	No	SMR metrics that depend on this metric. Used for impact analysis when metric definitions change.
`access.roles`	Yes	Role IDs from the entitlement config that may query this metric.

Dimension definition schema

```
dimension:
  id:          "asset_class"
  version:     "1.0.0"
  label:       "Asset Class"
  description: "Broad classification of securities by asset type."
  type:        "categorical"
  values:
    source:    "reference_data"
    endpoint:  "dim_asset_class"
    cacheable: true
  cardinality: "low"
  governance:
    owner:          "data_management_office"
    classification: "PUBLIC"
    approved:       true
  display:
    sort_order: "label_ascending"
    default_filter: null
```

Field	Description
`type`	`categorical`, `temporal`, `numeric_range`, `geographic`, `hierarchical_node`
`cardinality`	`low` (< 20 values), `medium` (20–500), `high` (500+), `unbounded`. Used by the FQP for result size estimation.

Hierarchy definition schema

```
hierarchy:
  id:          "asset_class_hierarchy"
  version:     "1.0.0"
  label:       "Asset Class Hierarchy"
  description: "Navigable hierarchy from broad asset class through sub-asset class to security
type."
  levels:
    - id:    "asset_class"
      label: "Asset Class"
      depth: 1
    - id:    "sub_asset_class"
      label: "Sub-Asset Class"
      depth: 2
    - id:    "security_type"
      label: "Security Type"
      depth: 3
  drilldown:
    enabled:   true
    max_depth: 3
  governance:
    owner:     "data_management_office"
    approved:  true
```

Measure Group definition schema

```
measure_group:
  id:          "performance_metrics"
  label:       "Performance Metrics"
  description: "Standard portfolio performance measures for client reporting and investment
committee use."
  metrics:
    - "portfolio_return"
    - "benchmark_return"
    - "active_return"
    - "tracking_error"
    - "information_ratio"
    - "sharpe_ratio"
  default_dimensions:
    - "portfolio"
    - "date"
  governance:
    owner:    "head_of_performance_analytics"
    approved: true
```

Formula language

The SMR formula language is used to define metric computation logic. It references: - Other SMR metrics (by `id`) - Canonical data source identifiers (abstract references resolved by the FQP to physical columns) - Standard mathematical and financial functions from the platform's function library

Formula language examples:

```
# Simple ratio metric
active_return = portfolio_return - benchmark_return

# Information Ratio
information_ratio = active_return / tracking_error

# Weighted average with null protection
weighted_avg_duration = SAFE_DIVIDE(
  SUM(position_market_value * security_modified_duration),
  SUM(position_market_value)
)

# Conditional metric
issuer_concentration = SAFE_DIVIDE(
  SUM(position_market_value, FILTER(issuer = {{dim.issuer}})),
  SUM(position_market_value)
)

# Time-windowed metric
rolling_90d_return = CUMULATIVE_RETURN(daily_return, WINDOW(90, 'days'))
```

Built-in function library (selected):

Function	Description
`SUM(metric, [filter])`	Aggregated sum with optional dimensional filter
`SAFE_DIVIDE(numerator, denominator)`	Division returning null (not error) when denominator is zero
`CUMULATIVE_RETURN(period_return, window)`	Compounded return over a time window
`PERCENTILE(metric, pct)`	Percentile aggregation
`CONDITIONAL(condition, true_val, false_val)`	Conditional value selection
`BENCHMARK_RELATIVE(metric, benchmark_id)`	Metric expressed relative to a benchmark dimension
`WINDOW(n, unit)`	Defines a sliding time window
`DATE_TRUNC(dimension, granularity)`	Truncates a temporal dimension to a specified granularity

---

Registry governance workflow

```
Draft → Proposed → In Review → Approved (Active) → Deprecated → Retired
```

Transition	Trigger	Effect
Draft → Proposed	Metric owner submits a new definition	Visible in admin UI; not resolvable
Proposed → In Review	Application Admin opens for review	Downstream impact analysis runs automatically
In Review → Approved	Application Admin approves	Metric becomes resolvable from next refresh cycle
Approved → Deprecated	Owner or Admin marks deprecated	Metric resolves with a deprecation warning; removed from SMR browsing defaults
Deprecated → Retired	Admin retires after deprecation period	Metric no longer resolvable; lineage records preserved

Impact analysis on metric change

When a metric definition is proposed for change, the platform automatically runs an impact analysis:

1. Identify all downstream metrics referencing this metric via `lineage.upstream_metrics`
2. Identify all saved analytical sessions and scheduled queries using this metric
3. Identify all dashboards and visualisations embedding results derived from this metric
4. Produce an impact report for the Application Admin to review before approving the change

Approval of a change that has downstream impacts requires the Application Admin to acknowledge the impact report. This acknowledgement is recorded in the lineage store.

SMR API

The SMR is accessible via the Platform Admin API and via the MCP Capability Layer (for AI agents):

```
GET  /v1/smr/metrics              — list all approved metrics
GET  /v1/smr/metrics/{id}         — get metric definition by ID
GET  /v1/smr/metrics/{id}/lineage — get full lineage graph for metric
GET  /v1/smr/dimensions           — list all approved dimensions
GET  /v1/smr/hierarchies          — list all approved hierarchies
GET  /v1/smr/measure-groups       — list all measure groups
POST /v1/smr/metrics              — propose a new metric definition
PUT  /v1/smr/metrics/{id}         — propose a change to an existing metric
POST /v1/smr/metrics/{id}/approve — approve a proposed metric (Application Admin only)
```

Registry browsing in the UI

Authenticated users (within their entitlement scope) can browse the SMR from the analytics interface:

- Searchable metric catalogue with full definitions, formulas, owners, and lineage
- Dimension reference with value previews
- Hierarchy visualisation showing navigable levels
- Measure group collections for common analytical workflows
- Metric change history and version comparison

Users may only browse metrics within their `access.roles` entitlement. Metrics they are not entitled to are not visible in the browser.

05 — Analytics DSL

Overview

The **Analytics DSL** is the platform's constrained query language — the interface layer between analytical intent and the Federated Query Planner. It exposes business semantics drawn from the Semantic Metrics Registry, supports AI reasoning about analytical queries, and compiles to engine-agnostic Logical Query Plans (LQPs).

The DSL is not exposed directly to end users. It is the internal representation produced by the Semantic Intent Layer after resolving natural language against the SMR. It may also be used directly by AI orchestrators accessing the platform via the MCP Capability Layer.

The DSL deliberately excludes: - Physical table or column references - Database-specific syntax or functions - JOIN operations (joins are resolved by the FQP using data affinity declarations) - Raw SQL passthrough of any kind

DSL grammar (EBNF notation)

```
query           ::= "ANALYSE" metric-list
                    ["BY" dimension-list]
                    ["WHERE" filter-expression]
                    ["FOR" time-expression]
                    ["ORDER BY" order-expression]
                    ["LIMIT" integer]
                    ["COMPARE TO" comparison-target]
                    ["DRILLDOWN" drilldown-expression]

metric-list     ::= metric-ref ("," metric-ref)*
metric-ref      ::= metric-id ["AS" alias]
                  | "MEASURE_GROUP" "(" measure-group-id ")"

dimension-list  ::= dimension-ref ("," dimension-ref)*
dimension-ref   ::= dimension-id ["AT" granularity]

filter-expr     ::= predicate ("AND" | "OR" predicate)*
predicate       ::= dimension-id comparison-op value
                  | metric-id comparison-op value
                  | metric-id "BETWEEN" value "AND" value
                  | predicate "AND" predicate
                  | predicate "OR" predicate
                  | "NOT" predicate
                  | "(" predicate ")"

time-expression ::= "PERIOD" "(" period-spec ")"
                  | "RANGE" "(" date-literal "TO" date-literal ")"
                  | "LAST" integer time-unit
                  | "YEAR_TO_DATE"
                  | "QUARTER_TO_DATE"
                  | "SINCE_INCEPTION"
                  | "FISCAL_YEAR" integer

comparison-op   ::= "=" | "!=" | ">" | "<" | ">=" | "<="
                  | "IN" "(" value-list ")" | "NOT IN" "(" value-list ")"
                  | "LIKE" string | "IS NULL" | "IS NOT NULL"

comparison-target ::= "BENCHMARK" "(" benchmark-id ")"
                    | "PERIOD" "(" period-spec ")"
                    | "PEER_GROUP" "(" peer-group-id ")"

drilldown-expr  ::= "INTO" dimension-id ["AT" depth]

order-expression ::= metric-id | dimension-id ("ASC" | "DESC")

granularity     ::= "DAY" | "WEEK" | "MONTH" | "QUARTER" | "YEAR"
time-unit       ::= "DAYS" | "WEEKS" | "MONTHS" | "QUARTERS" | "YEARS"
```

DSL examples

Example 1 — Single metric, single dimension, time filter

Natural language: "Show me portfolio returns for the current quarter."

Resolved DSL:

```
ANALYSE portfolio_return
BY portfolio
FOR QUARTER_TO_DATE
ORDER BY portfolio_return DESC
```

Example 2 — Multi-metric, multi-dimension, benchmark comparison

Natural language: "Show portfolio return and tracking error by asset class compared to benchmark."

Resolved DSL:

```
ANALYSE portfolio_return, tracking_error
BY portfolio, asset_class
FOR QUARTER_TO_DATE
COMPARE TO BENCHMARK(b_msci_acwi)
ORDER BY tracking_error DESC
```

Example 3 — Filtered query with threshold

Natural language: "Which portfolios have VaR exceeding their limit today?"

Resolved DSL:

```
ANALYSE var_95, var_limit, var_utilisation_pct
BY portfolio
FOR PERIOD(today)
WHERE var_utilisation_pct > 1.0
ORDER BY var_utilisation_pct DESC
```

Example 4 — Measure group with rolling time window

Natural language: "Show me performance metrics for the last 12 months."

Resolved DSL:

```
ANALYSE MEASURE_GROUP(performance_metrics)
BY portfolio, date AT MONTH
FOR LAST 12 MONTHS
ORDER BY date ASC
```

Example 5 — Drilldown expression

Natural language: "Drill into asset class for the Global Equity portfolio — show me sub-asset class breakdown."

Resolved DSL:

```
ANALYSE portfolio_return, aum
BY sub_asset_class
FOR QUARTER_TO_DATE
WHERE portfolio = 'GLOBAL_EQUITY_OPPORTUNITIES'
DRILLDOWN INTO asset_class_hierarchy AT 2
```

Example 6 — Issuer concentration with threshold filter

Natural language: "Show issuers where concentration exceeds 5% of AUM across all equity portfolios."

Resolved DSL:

```
ANALYSE issuer_concentration, aggregate_position_value
BY issuer, portfolio
FOR PERIOD(today)
WHERE asset_class = 'EQUITY'
  AND issuer_concentration > 0.05
ORDER BY issuer_concentration DESC
```

DSL compiler

The DSL compiler is the component that: 1. Receives a DSL expression (produced by the Semantic Intent Layer from natural language) 2. Validates all identifiers against the active SMR for the tenant 3. Applies role-aware projection (see 09-role-aware-projections.md) 4. Produces an engine-agnostic Logical Query Plan (LQP)

Compilation stages

Compiler error types

Error type	Example	User-facing message
`METRIC_NOT_FOUND`	Metric ID not in SMR	*"The metric 'xyz' is not defined in the analytics registry."*
`METRIC_NOT_ENTITLED`	Metric in SMR but not in user's role	*"You do not have access to the metric 'portfolio_return'."*
`REQUIRED_DIMENSION_MISSING`	`portfolio_return` queried without `portfolio`	*"The metric 'portfolio_return' requires the 'portfolio' dimension."*
`INVALID_AGGREGATION`	Aggregation not in `allowed` list	*"The aggregation 'count' is not supported for metric 'portfolio_return'."*
`UNSUPPORTED_GRANULARITY`	Daily granularity for a monthly-only metric	*"The metric 'monthly_nav' is only calculable at monthly or lower frequency."*
`SYNTAX_ERROR`	Malformed DSL expression	*"Analytical query could not be parsed — please rephrase your question."*
`HIERARCHY_NOT_ALLOWED`	Drilldown hierarchy not in tenant config	*"Drilldown into 'counterparty_hierarchy' is not enabled for this application."*

Logical Query Plan (LQP) schema

The LQP is a directed acyclic graph of analytical operations, serialised as JSON:

```
{
  "lqp_id":      "lqp_20260514_093247_a1b2c3",
  "tenant_id":   "acme-wealth",
  "version":     "1.0",
  "created_at":  "2026-05-14T09:32:47Z",
  "dsl_source":  "ANALYSE portfolio_return, tracking_error BY portfolio FOR QUARTER_TO_DATE",
  "metrics": [
    {
      "id":          "portfolio_return",
      "version":     "2.1.0",
      "aggregation": "value_weighted_average",
      "period":      "quarter_to_date"
    },
    {
      "id":          "tracking_error",
      "version":     "1.3.0",
      "aggregation": "value_weighted_average",
      "period":      "quarter_to_date"
    }
  ],
  "dimensions": [
    { "id": "portfolio", "required": true }
  ],
  "filters": [
    {
      "type":      "row_predicate",
      "source":    "role_projection",
      "predicate": "portfolio_id IN ('GLOB_EQ_OPP', 'UK_CORE_INC', 'STRAT_BAL')"
    }
  ],
  "time": {
    "type":        "period",
    "period":      "quarter_to_date",
    "as_of_date":  "2026-05-14"
  },
  "data_affinity": {
    "portfolio_return": "portfolio",
    "tracking_error":   "risk_metrics"
  },
  "cost_estimate": {
    "units":       450,
    "confidence":  "medium"
  },
  "cardinality_estimate": {
    "rows":        14,
    "columns":     3
  }
}
```

AI interaction with the DSL

The Analytics DSL is the primary interface for AI agents accessing the platform via the MCP Capability Layer. When an AI orchestrator submits an analytical query, it may do so via:

1. **Natural language** — the Semantic Intent Layer resolves to DSL internally
2. **DSL directly** — the AI submits a DSL expression to the `/v1/query/dsl` endpoint

3. **Pre-defined analytical operations** — the AI invokes named operations from the MCP Capability Layer (which internally produce DSL)

In all cases, the DSL passes through the compiler pipeline — there is no "trusted" bypass for AI-submitted queries. AI agents are subject to the same SMR validation, role-aware projection, and governance checks as human users.

06 — Federated Query Planning

Overview

The **Federated Query Planner (FQP)** is the component that receives a validated Logical Query Plan (LQP) from the Analytics DSL compiler, decomposes it into engine-specific sub-plans, routes sub-plans to registered execution engines, manages result assembly, and handles caching and materialisation.

The FQP is the only component in the platform that has knowledge of physical execution engines. No other component — not the DSL compiler, not the Semantic Intent Layer, not the AI model — has access to engine connection details or physical schema information.

FQP architecture

Sub-plan decomposition

The FQP decomposes an LQP into sub-plans by **data affinity** — the logical data domain declared by each metric in its SMR definition.

Example LQP decomposition:

LQP requesting `portfolio_return` (affinity: `portfolio`) and `var_95` (affinity: `risk_metrics`) by `portfolio` and `date`:

```
LQP
├── Sub-plan A: portfolio_return, aum BY portfolio, date
│   └── Affinity: portfolio → Engine: snowflake-primary
│
└── Sub-plan B: var_95, var_limit BY portfolio, date
    └── Affinity: risk_metrics → Engine: dbt-semantic
```

The shared dimensions (`portfolio`, `date`) become the join keys for result assembly.

Sub-plan priority resolution

When multiple engines are registered for the same data affinity, the FQP selects the highest-priority available engine (lowest `priority` number in the config). If the highest-priority engine is unavailable or times out, the FQP automatically falls back to the next registered engine for the same affinity.

Physical query generation

The FQP translates sub-plans into engine-specific query dialects. This is the only point in the platform where engine-specific syntax is generated:

Snowflake dialect example

Sub-plan (LQP fragment):

```
{
  "metrics": ["portfolio_return"],
  "dimensions": ["portfolio"],
  "time": { "period": "quarter_to_date", "as_of_date": "2026-05-14" },
  "filters": [
    { "predicate": "portfolio_id IN ('GLOB_EQ_OPP', 'UK_CORE_INC')" }
  ]
}
```

Generated Snowflake SQL (internal, never exposed to AI or user):

```
SELECT
  p.portfolio_id,
  p.portfolio_name,
  SUM(pd.market_value * pd.daily_return) / SUM(pd.market_value) AS portfolio_return
FROM fact_portfolio_daily pd
JOIN dim_portfolio p ON pd.portfolio_id = p.portfolio_id
WHERE pd.price_date >= DATE_TRUNC('quarter', CURRENT_DATE)
  AND pd.price_date <= '2026-05-14'
  AND pd.portfolio_id IN ('GLOB_EQ_OPP', 'UK_CORE_INC')
GROUP BY p.portfolio_id, p.portfolio_name
ORDER BY p.portfolio_name ASC
```

dbt Semantic Layer dialect example

Sub-plan (LQP fragment):

```
{
  "metrics": ["var_95"],
  "dimensions": ["portfolio"],
  "time": { "period": "today", "as_of_date": "2026-05-14" }
}
```

Generated dbt Semantic Layer query:

```
{
  "metrics": [{ "name": "var_95" }],
  "group_by": [{ "name": "Dimension('portfolio__portfolio_id')" }],
  "where": [{ "sql": "{{ Dimension('portfolio__portfolio_id') }} IN ('GLOB_EQ_OPP',
'UK_CORE_INC')" }],
  "limit": 1000
}
```

Cube OLAP dialect example

Sub-plan (LQP fragment):

```
{
  "metrics": ["aum"],
  "dimensions": ["portfolio", "asset_class"],
  "time": { "period": "quarter_to_date" }
}
```

Generated Cube query:

```
{
  "measures": ["Portfolios.aum"],
  "dimensions": ["Portfolios.portfolioId", "Portfolios.portfolioName",
"Securities.assetClass"],
  "timeDimensions": [{
    "dimension": "PortfolioDaily.priceDate",
    "dateRange": "this quarter"
  }],
  "filters": [
    {
      "member": "Portfolios.portfolioId",
      "operator": "equals",
      "values": ["GLOB_EQ_OPP", "UK_CORE_INC"]
    }
  ]
}
```

Result assembly

The FQP assembles sub-results from multiple engines into a unified result set:

1. **Join key identification** — the shared dimensions across sub-plans (e.g. `portfolio_id`, `date`)
2. **In-memory join** — sub-results are joined in the FQP result assembler by shared keys. Physical joins across engines do not occur at the engine level.
3. **Missing value handling** — if a sub-result has no row for a shared key that exists in another sub-result, the missing metric values are represented as null with a provenance marker in the lineage record.
4. **Column mask application** — column masks declared in the Role-Aware Projection are applied to the assembled result before it leaves the FQP.
5. **Result validation** — assembled result is validated against the LQP specification (correct columns, expected cardinality range, data type conformance).

Caching and materialisation

Result cache

The FQP maintains a result cache keyed by the LQP signature (a deterministic hash of the metric set, dimensions, filters, time expression, and entitlement hash).

Cache behaviour	Specification
Cache key	SHA-256 of (metric IDs + versions, dimension IDs, filter predicates, time expression, entitlement hash, tenant ID)
Cache TTL	Configurable per `data.refresh_cadence` in the metric definition. Default: `3600` seconds.
Cache invalidation	On metric definition version change; on execution engine data refresh signal (if the engine emits one); on explicit cache clear via Admin API
Cache scope	Per-tenant. Results from one tenant are never served to another.
Cache storage	Platform-managed Redis-compatible store. Results over 10MB bypass the cache and are streamed directly.
Cache hit disclosure	Cache hits are disclosed in the lineage record and (optionally) to the user as a *"Result from cache (data as of [timestamp])"* indicator.

Materialised views

For high-frequency queries that consistently resolve to the same sub-plans, the FQP supports **materialised view registration** via the Admin API:

```
{
  "materialised_view": {
    "id":            "portfolio_qtd_summary",
    "description":   "Pre-materialised portfolio QTD return and tracking error summary",
    "lqp_template":  { ... },
    "refresh_schedule": "0 6 * * *",
    "engines":       ["snowflake-primary"],
    "cost_offset":   -800
  }
}
```

Materialised views reduce the cost unit estimate for matching sub-plans, allowing queries that would otherwise trigger a cost circuit breaker to execute against the pre-materialised result.

Adaptive planning

The FQP adapts routing decisions based on observed execution performance:

Adaptive behaviour	Description
Engine latency tracking	The FQP tracks p50/p95 latency per engine per data affinity over a rolling 1-hour window
Automatic fallback	If an engine's p95 latency exceeds twice its baseline, the FQP automatically routes to the next available engine for new queries
Cost estimate calibration	The FQP updates cost unit estimates based on observed execution cost data from completed queries
Partial result handling	If a sub-plan engine returns a partial result (e.g. timeout with partial rows), the FQP logs the partial result in the lineage record and surfaces a *"Result may be incomplete — [engine] timed out"* warning to the user

---

FQP execution record (in lineage)

The FQP writes a complete execution record to the lineage store for every query:

```
{
  "fqp_execution_id": "fqp_exec_20260514_093247",
  "lqp_id":           "lqp_20260514_093247_a1b2c3",
  "cache_hit":        false,
  "sub_plans": [
    {
      "id":             "sp_a",
      "engine_id":      "snowflake-primary",
      "metrics":        ["portfolio_return"],
      "status":         "success",
      "latency_ms":     1240,
      "rows_returned":  14,
      "cost_units":     320
    },
    {
      "id":             "sp_b",
      "engine_id":      "dbt-semantic",
      "metrics":        ["tracking_error"],
      "status":         "success",
      "latency_ms":     890,
      "rows_returned":  14,
      "cost_units":     180
    }
  ],
  "assembly_latency_ms": 45,
  "total_latency_ms":    1285,
  "total_cost_units":    500,
  "result_rows":         14,
  "result_columns":      3,
  "cache_written":       true,
  "cache_ttl_seconds":   3600
}
```

07 — Visualisation Ontology

Overview

The **Visualisation Ontology** is the governing schema that maps result characteristics and analytical intent patterns to specific, parameterised chart contracts. It ensures that the same analytical pattern always produces the same chart type — across users, sessions, and time — regardless of which AI model is in use or how the question was phrased.

The Visualisation Ontology makes chart selection **deterministic and governed**. The AI model does not select chart types. It may express an intent preference (e.g. *"show me a breakdown"*), which the ontology treats as an intent signal alongside the result schema — but the ontology makes the final binding decision.

Chart contract model

Each entry in the Visualisation Ontology is a **chart contract** — a named, versioned specification that defines:

1. **Match conditions** — what result schema and intent pattern this contract applies to
2. **Chart type** — the chart class to render
3. **Axis mappings** — how result columns map to visual axes and encodings
4. **Interaction semantics** — what click, hover, and drilldown interactions are available
5. **Rendering parameters** — default visual parameters (colours, labels, thresholds)

Intent pattern taxonomy

The ontology classifies every analytical result into one of seven intent patterns, derived from the DSL expression and result schema:

Intent pattern	Description	Typical trigger phrases
`COMPARISON`	Comparing a metric across discrete categories	"compare", "by", "across", "ranked by", "top N"
`TREND`	Showing how a metric changes over time	"over time", "trend", "history", "30-day", "since"
`DISTRIBUTION`	Showing the spread or concentration of a metric	"distribution", "breakdown", "composition", "concentration"
`THRESHOLD`	Comparing a metric against a limit or benchmark	"exceeding", "within limit", "versus benchmark", "breach"
`ATTRIBUTION`	Decomposing a metric into contributing factors	"attribution", "contribution", "drivers", "breakdown by"

Intent pattern	Description	Typical trigger phrases
`RELATIONSHIP`	Showing correlation or dependency between metrics	"versus", "correlation", "scatter", "risk-return"
`COMPOSITION`	Showing part-to-whole relationships	"proportion", "weight", "allocation", "share"

Ontology contract definitions

Contract 1 — Multi-series bar (COMPARISON)

Contract ID: `BAR_MULTI_SERIES_COMPARISON`

Match conditions: - Intent pattern: `COMPARISON` - Metric count: 1–3 - Primary dimension: categorical, cardinality: low–medium (< 50) - Time dimension: absent OR single point in time

Axis mappings: - X-axis: primary categorical dimension (sorted by primary metric DESC by default) - Y-axis: metric value - Colour encoding: metric series (for multi-metric) OR secondary dimension if present - Tooltip: all metrics for the hovered category

Example Vega-Lite contract (abbreviated):

```
{
  "contract_id":    "BAR_MULTI_SERIES_COMPARISON",
  "version":        "1.2.0",
  "chart_type":     "bar",
  "library":        "vega-lite",
  "spec_template": {
    "$schema": "https://vega.github.io/schema/vega-lite/v5.json",
    "mark":    { "type": "bar", "tooltip": true },
    "encoding": {
      "x": {
        "field":  "{{primary_dimension}}",
        "type":   "ordinal",
        "sort":   "-y",
        "axis":   { "labelAngle": -30 }
      },
      "y": {
        "field":    "{{primary_metric}}",
        "type":     "quantitative",
        "axis":     { "format": "{{metric_format}}" }
      },
      "color": {
        "field":    "{{series_dimension}}",
        "type":     "nominal",
        "scale":    { "scheme": "{{color_palette}}" }
      }
    }
  },
  "interaction": {
    "click":       "drilldown",
    "hover":       "tooltip",
    "selection":   "multi_point"
  }
}
```

Contract 2 — Time series line (TREND)

Contract ID: `LINE_TIME_SERIES_TREND`

Match conditions: - Intent pattern: `TREND` - Time dimension: present, cardinality: multiple points - Metric count: 1–5

Axis mappings: - X-axis: temporal dimension - Y-axis: metric value - Colour: metric series or secondary categorical dimension - Reference line: injected if `COMPARE TO BENCHMARK` present in DSL

Interaction semantics: - Hover: crosshair tooltip showing all series values at hovered date - Click on a data point: surface lineage for that specific time period - Brush selection: zooms X-axis to selected range

---

Contract 3 — Heatmap (THRESHOLD)

Contract ID: `HEATMAP_THRESHOLD_MATRIX`

Match conditions: - Intent pattern: `THRESHOLD` - Two categorical dimensions present - Metric count: 1 (the threshold metric) - Threshold value available (from `var_limit`, `budget`, etc.)

Axis mappings: - X-axis: first categorical dimension (e.g. date) - Y-axis: second categorical dimension (e.g. portfolio) - Colour encoding: metric value expressed as % of threshold (diverging colour scale) - Threshold boundary: colour scale midpoint anchored at 100% of limit

Interaction semantics: - Click on cell: drilldown into that dimension intersection - Colour scale: red (> 100% limit) → amber (80–100%) → green (< 80%)

Contract 4 — Treemap (COMPOSITION)

Contract ID: `TREEMAP_COMPOSITION`

Match conditions: - Intent pattern: `COMPOSITION` OR `DISTRIBUTION` - One categorical dimension, cardinality: medium–high - One metric representing a positive quantity (AUM, market value, weight)

Axis mappings: - Area: proportional to metric value - Colour: secondary metric (e.g. return) — diverging scale - Label: dimension value + metric value

Interaction semantics: - Click on tile: drilldown into next hierarchy level - Hover: tooltip showing all metrics for the tile

---

Contract 5 — Waterfall (ATTRIBUTION)

Contract ID: `WATERFALL_ATTRIBUTION`

Match conditions: - Intent pattern: `ATTRIBUTION` - Metrics include a total metric and one or more contribution metrics - Contribution metrics sum to total metric

Axis mappings: - X-axis: contribution dimension (factors, asset classes, etc.) - Y-axis: contribution value (positive and negative) - Colour: positive contribution (green), negative contribution (red), total (grey)

Contract 6 — Scatter / risk-return (RELATIONSHIP)

Contract ID: `SCATTER_RISK_RETURN`

Match conditions: - Intent pattern: `RELATIONSHIP` - Exactly two numeric metrics - One categorical dimension (e.g. portfolio, asset class)

Axis mappings: - X-axis: first metric (conventionally risk measure) - Y-axis: second metric (conventionally return measure) - Colour: categorical dimension - Size: optional third metric (e.g. AUM) - Reference lines: quadrant boundaries from benchmark values (if present)

Contract 7 — Data table (fallback and high-cardinality)

Contract ID: `TABLE_GOVERNED`

Match conditions (any of): - Metric count > 5 - Primary dimension cardinality: high (> 50) - No clear intent pattern match - `LIMIT` clause present with no ordering by a visualisable metric - User explicitly requested tabular output

Table features: - Column headers from SMR metric labels and dimension labels - Column-level sorting - Column-level filtering - Number formatting from metric `display.format` and `display.decimals` - Inline sparklines for temporal metrics - Export to CSV and JSON

Ontology evaluation algorithm

The ontology evaluator receives: 1. The LQP (metrics, dimensions, time, filters) 2. The intent pattern classification (from the Semantic Intent Layer) 3. The result schema (actual column types and cardinalities from the FQP result)

It evaluates contracts in order of specificity and returns the highest-specificity matching contract:

```
def evaluate_ontology(lqp, intent_pattern, result_schema, allowed_charts):
    candidates = []
    for contract in ONTOLOGY_CONTRACTS:
        if not all(c in allowed_charts for c in [contract.chart_type]):
            continue
        score = contract.match_score(lqp, intent_pattern, result_schema)
        if score > 0:
            candidates.append((score, contract))
    candidates.sort(key=lambda x: x[0], reverse=True)
    if candidates:
        return candidates[0][1]
    else:
        return TABLE_GOVERNED  # deterministic fallback
```

Chart ontology override

Power Analysts may override the ontology's chart selection for a single result via an explicit intent in their query:

```
Show me portfolio returns as a line chart over the last 12 months
```

The override is logged in the lineage record as an analyst-requested deviation from the governing ontology. Overrides are subject to: - The chart type being in the tenant's `allowedChartTypes` list - The result schema being compatible with the requested chart type (incompatible overrides are rejected with an explanation)

Rendering pipeline

After the ontology selects a chart contract, the rendering pipeline:

1. Populates the contract's `spec_template` with values derived from the result schema (field names, metric formats, axis labels from SMR labels)
2. Applies the tenant's colour palette and number format settings
3. Injects benchmark reference lines or threshold markers if present in the LQP
4. Produces a final chart spec (Vega-Lite, Plotly, or ECharts JSON) ready for client rendering
5. Produces a data table as a secondary view for all chart types (always available via a "Show as table" toggle)

Narrative anchoring to visualisation

The Narrative Synthesis Engine receives: - The chart contract selected by the ontology - The chart's axis mappings (so it knows what is on each axis) - The result set values - The SMR metric definitions (label, unit, description)

It uses this structured context to produce prose that directly references the chart's visual elements: *"The chart shows quarterly return on the Y-axis against portfolio on the X-axis, sorted by highest return. Global Equity Opportunities leads at 4.2%, followed by..."*

The narrative never introduces values or comparisons not visible in the chart or derivable from the result set.

08 — MCP Capability Layer

Overview

The **MCP Capability Layer** exposes the AI Analytics Platform's governed analytical operations to AI orchestrators via an MCP-compatible interface. It is the primary integration point for AI agents — including the platform's own Semantic Intent Layer — that need to compose analytical queries programmatically.

The MCP Capability Layer does not expose raw query interfaces. Every capability is a bounded, named analytical operation that enforces the same governance pipeline as natural-language queries. AI agents interact with analytical capabilities, not with databases.

Capability model

Each MCP capability is a named analytical operation with: - A typed input schema (parameters the AI agent provides) - A governed execution path (routes through DSL compiler, projection layer, governance, FQP) - A typed output contract (the result schema the agent can expect) - A semantic description (the context injected into the AI model when this capability is available)

Core analytical capabilities

`analyse_metric`

Execute a governed analytical query against one or more registered metrics.

```
{
  "name":        "analyse_metric",
  "description": "Execute a governed analytical query against one or more registered metrics
from the Semantic Metrics Registry. Use this to answer quantitative questions about portfolio
performance, risk, or other financial metrics. Always specify the metric IDs you need — do not
attempt to construct SQL or use unregistered identifiers.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "metrics": {
        "type":        "array",
        "items":       { "type": "string" },
        "description": "Array of metric IDs from the Semantic Metrics Registry (e.g.
['portfolio_return', 'tracking_error'])"
      },
      "dimensions": {
        "type":        "array",
        "items":       { "type": "string" },
        "description": "Dimension IDs to slice by (e.g. ['portfolio', 'asset_class'])"
      },
      "time_period": {
        "type":        "string",
        "description": "Time expression. Accepted: 'quarter_to_date', 'year_to_date',
'last_N_months', 'since_inception', 'today', or 'RANGE:YYYY-MM-DD:YYYY-MM-DD'"
      },
      "filters": {
        "type":        "array",
        "items": {
          "type":      "object",
          "properties": {
            "dimension": { "type": "string" },
            "operator":  { "type": "string", "enum": ["eq", "neq", "gt", "lt", "gte", "lte",
"in", "not_in"] },
            "value":     {}
          }
        }
      },
      "order_by": {
        "type":        "string",
        "description": "metric_id ASC|DESC"
      },
      "limit": {
        "type":        "integer",
        "description": "Maximum rows to return. Default: 1000."
      }
    },
    "required": ["metrics", "time_period"]
  }
}
```

Example invocation:

```
{
  "metrics":     ["portfolio_return", "tracking_error"],
  "dimensions":  ["portfolio"],
  "time_period": "quarter_to_date",
  "filters": [
    { "dimension": "asset_class", "operator": "eq", "value": "EQUITY" }
  ],
  "order_by":    "tracking_error DESC"
}
```

`compare_portfolios`

Compare a set of metrics across two or more portfolios, optionally against a benchmark.

```
{
  "name":        "compare_portfolios",
  "description": "Compare governed metrics across multiple portfolios. Optionally includes
benchmark comparison. Returns a structured comparison result suitable for tabular or chart
rendering.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "portfolio_ids": {
        "type":  "array",
        "items": { "type": "string" },
        "description": "Portfolio identifiers to compare"
      },
      "metrics": {
        "type":  "array",
        "items": { "type": "string" }
      },
      "time_period": { "type": "string" },
      "benchmark_id": {
        "type":        "string",
        "description": "Benchmark ID to compare against (optional)"
      }
    },
    "required": ["portfolio_ids", "metrics", "time_period"]
  }
}
```

`issuer_concentration`

Calculate issuer concentration metrics across specified portfolios.

```
{
  "name":        "issuer_concentration",
  "description": "Calculate issuer concentration risk metrics. Returns issuer-level exposure,
concentration percentage of total AUM, and limit utilisation where limits are defined. Use for
regulatory concentration checks and investment committee reporting.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "portfolio_ids":      { "type": "array", "items": { "type": "string" } },
      "asset_class_filter": { "type": "string", "description": "Optional asset class
filter" },
      "threshold_pct":      { "type": "number",  "description": "Highlight issuers above this
concentration percentage (0–1)" },
      "as_of_date":         { "type": "string",  "format": "date" }
    },
    "required": ["portfolio_ids", "as_of_date"]
  }
}
```

`risk_breakdown`

Decompose risk metrics into contributing factors.

```
{
  "name":        "risk_breakdown",
  "description": "Decompose a risk metric (VaR, tracking error, beta) into factor
contributions. Returns a structured attribution of the total risk figure to its contributing
dimensions. Use for risk reporting and limit breach investigation.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "portfolio_id":  { "type": "string" },
      "risk_metric":   { "type": "string", "description": "Risk metric ID (e.g. 'var_95',
'tracking_error')" },
      "attribution_by": {
        "type":        "string",
        "enum":        ["asset_class", "factor", "issuer", "geography", "currency"],
        "description": "Dimension to attribute risk contribution to"
      },
      "as_of_date": { "type": "string", "format": "date" }
    },
    "required": ["portfolio_id", "risk_metric", "attribution_by", "as_of_date"]
  }
}
```

`performance_attribution`

Brinson-Hood-Beebower (or Brinson-Fachler) performance attribution decomposition.

```
{
  "name":        "performance_attribution",
  "description": "Run a governed performance attribution decomposition (BHB or BF model) for a
portfolio versus its benchmark. Returns allocation effect, selection effect, and interaction
effect by dimension. Use for investment committee reporting and mandate compliance.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "portfolio_id":    { "type": "string" },
      "benchmark_id":    { "type": "string" },
      "attribution_by":  { "type": "string", "enum": ["asset_class", "geography", "sector",
"currency"] },
      "time_period":     { "type": "string" },
      "model":           { "type": "string", "enum": ["bhb", "bf"], "description":
"Attribution model. Default: bhb" }
    },
    "required": ["portfolio_id", "benchmark_id", "attribution_by", "time_period"]
  }
}
```

`regulatory_metric`

Query a regulatory compliance metric.

```
{
  "name":        "regulatory_metric",
  "description": "Query a regulatory compliance metric (LCR, NSFR, leverage ratio, etc.).
Requires the regulatory_reporting feature flag and appropriate role. Returns the metric value,
regulatory minimum, and compliance status.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "metric_id":   { "type": "string", "description": "Regulatory metric ID from the SMR" },
      "entity_ids":  { "type": "array", "items": { "type": "string" } },
      "as_of_date":  { "type": "string", "format": "date" },
      "trend_days":  { "type": "integer", "description": "Number of trailing days to include
as trend. 0 for current only." }
    },
    "required": ["metric_id", "entity_ids", "as_of_date"]
  }
}
```

`list_metrics`

Retrieve the list of metrics available to the current user from the SMR.

```
{
  "name":        "list_metrics",
  "description": "List all metrics in the Semantic Metrics Registry available to the current
user based on their role. Use this to understand what analytical concepts are resolvable
before attempting a query. Returns metric IDs, labels, descriptions, and required
dimensions.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "domain_filter":    { "type": "string", "description": "Optional: filter to a specific
data domain" },
      "measure_group":    { "type": "string", "description": "Optional: filter to a named
measure group" },
      "include_deprecated": { "type": "boolean", "default": false }
    }
  }
}
```

`drilldown`

Navigate into a dimension hierarchy from a previous result.

```
{
  "name":        "drilldown",
  "description": "Navigate into an analytical hierarchy from a prior result. Takes the result
set ID and a dimension to drill into, and returns a new result at the next hierarchy level
with the parent-level filters preserved.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "result_id":       { "type": "string", "description": "The result set ID from a prior
analyse_metric call" },
      "drilldown_into":  { "type": "string", "description": "Hierarchy ID to traverse into" },
      "selected_value":  { "description": "The dimension value to anchor the drilldown to
(e.g. 'EQUITY')" }
    },
    "required": ["result_id", "drilldown_into"]
  }
}
```

Capability manifest

The MCP Capability Layer exposes a capability manifest that AI orchestrators retrieve to understand what operations are available for the authenticated user and tenant:

```
{
  "manifest_version": "1.0",
  "tenant_id":        "acme-wealth",
  "as_of":            "2026-05-14T09:00:00Z",
  "capabilities": [
    { "name": "analyse_metric",          "available": true,  "reason": null },
    { "name": "compare_portfolios",      "available": true,  "reason": null },
    { "name": "issuer_concentration",    "available": true,  "reason": null },
    { "name": "risk_breakdown",          "available": true,  "reason": null },
    { "name": "performance_attribution", "available": true,  "reason": null },
    { "name": "regulatory_metric",       "available": false, "reason": "regulatory_reporting
feature flag disabled" },
    { "name": "list_metrics",            "available": true,  "reason": null },
    { "name": "drilldown",               "available": true,  "reason": null }
  ],
  "entitlement_summary": {
    "role":            "portfolio_manager",
    "accessible_metrics": 42,
    "accessible_dimensions": 12
  }
}
```

Capability invocation governance

Every MCP capability invocation passes through the full governance pipeline:

```
Capability invocation
→ Input schema validation
→ Capability availability check (feature flags + role)
→ DSL expression construction from capability inputs
→ DSL compiler (SMR resolution + role-aware projection)
→ Semantic Execution Governance
→ Federated Query Planner
→ Result assembly
→ Lineage record writing
→ Result returned to AI agent
```

AI agents receive the same lineage-backed, role-aware, governance-validated results as human users. There is no privileged API path.

MCP endpoint

The MCP Capability Layer is exposed at:

```
POST https://api.analytics-platform.io/v1/mcp
     Authorization: Bearer {user_jwt}
     Content-Type: application/json
```

Following the MCP Streamable HTTP transport specification. Tool call results are returned as MCP `tool_result` content blocks with the result set as structured JSON.

09 — Role-Aware Projections

Overview

The **Role-Aware Projection Layer** is the component that applies the authenticated user's entitlement model to the resolved analytical intent before any query plan is compiled. It is the semantic-layer enforcement of data access controls — operating above the physical execution layer, before any query reaches an execution engine.

Role-aware projection is not optional and not bypassable. Every analytical request — whether from natural language, DSL, or MCP capability — passes through the projection layer.

Projection model

The projection layer applies four categories of restriction:

Restriction type	Description	Applied at
Metric access filter	Removes metrics from the resolved intent that the user's role is not entitled to query	DSL compilation — Stage 3
Dimension access filter	Removes dimensions the user is not entitled to slice by	DSL compilation — Stage 3
Row predicate injection	Injects SQL-like predicates that restrict which data rows the user can access	FQP physical query generation
Column mask application	Replaces or nullifies column values the user is not permitted to see in the assembled result	FQP result assembly

---

Projection lifecycle

```
Authenticated request arrives with JWT
│
├── 1. JWT validation (signature, expiry, tenant claim)
│
├── 2. Role claim extraction
│       roleClaimField: "analytics_roles"
│       extracted roles: ["portfolio_manager"]
│
├── 3. Entitlement profile construction
│       Merge all role definitions for the user's roles
│       Produce: metric_access_set, dimension_access_set,
│               row_predicates[], column_masks[]
│
├── 4. Metric access filter
│       Intersect requested metrics with metric_access_set
│       Unentitled metrics → METRIC_NOT_ENTITLED error (per metric)
│
├── 5. Dimension access filter
│       Intersect requested dimensions with dimension_access_set
│       Unentitled dimensions → DIMENSION_NOT_ENTITLED error (per dimension)
│
├── 6. Row predicate construction
│       Resolve predicate templates: {{user.managed_portfolios}}
│       → "portfolio_id IN ('GLOB_EQ_OPP', 'UK_CORE_INC', 'STRAT_BAL')"
│       Predicates stored in LQP for FQP injection at execution time
│
├── 7. Column mask registration
│       Register masked columns in LQP metadata
│       FQP applies masks during result assembly
│
└── 8. Projected LQP produced → proceeds to governance validation
```

Multi-role entitlement merging

Users may hold multiple roles simultaneously. The projection layer merges role entitlements using **union semantics** — a user holding both `portfolio_manager` and `risk_officer` roles receives the union of both metric access sets and both dimension access sets.

Row predicates and column masks from multiple roles are intersected (most restrictive wins):

Entitlement type	Merge strategy	Rationale
Metric access	Union	A user entitled to a metric via any role may query it
Dimension access	Union	A user entitled to a dimension via any role may use it
Row predicates	Intersection (AND)	All predicates must be satisfied — most restrictive wins
Column masks	Union	A column masked by any role is masked for the user

Example multi-role merge:

User has roles: `portfolio_manager` + `risk_officer`

	portfolio_manager	risk_officer	Merged
Metric access	{portfolio_return, aum, tracking_error, sharpe_ratio}	{var_95, var_99, beta, duration, tracking_error}	{portfolio_return, aum, tracking_error, sharpe_ratio, var_95, var_99, beta, duration}
Dimension access	{portfolio, asset_class, date}	{portfolio, asset_class, issuer, rating, date}	{portfolio, asset_class, issuer, rating, date}
Row predicates	portfolio_id IN ({{user.managed_portfolios}})	*(none)*	portfolio_id IN ({{user.managed_portfolios}})
Column masks	*(none)*	*(none)*	*(none)*

Row predicate template resolution

Row predicate templates reference JWT claim values using `{{user.claim_name}}` syntax. Resolution occurs at query time using the authenticated user's current JWT:

Predicate template (in entitlement config):

```
portfolio_id IN ({{user.managed_portfolios}})
```

JWT claim:

```
{ "managed_portfolios": ["GLOB_EQ_OPP", "UK_CORE_INC", "STRAT_BAL"] }
```

Resolved predicate (injected into FQP physical queries):

```
portfolio_id IN ('GLOB_EQ_OPP', 'UK_CORE_INC', 'STRAT_BAL')
```

Supported predicate functions:

Function	Description
{{user.claim_name}}	Direct JWT claim value (string or array)
{{user.claim_name | upper}}	Claim value uppercased
{{user.claim_name | join(',')}}	Array claim joined as a CSV string
CURRENT_USER_ID()	Platform user ID — resolved at execution time
CURRENT_DATE()	Execution date — resolved at execution time

Column masking

Column masks are applied by the FQP during result assembly, after sub-results are returned from execution engines but before the result is returned to the calling layer.

Masking modes (configured per tenant):

Mode	Masked column representation
`null_replacement`	Column value replaced with `null`
`redacted_label`	Column value replaced with the string `"[REDACTED]"`
`excluded`	Column omitted entirely from the result schema

Example:

Compliance analyst role has `column_masks: ["client_name", "account_number"]` with `columnMaskingMode: "redacted_label"`.

Assembled result before masking:

```
[
  { "portfolio_id": "GLOB_EQ_OPP", "client_name": "Blackwood Family Trust", "account_number":
"WM-00412", "lcr": 1.24 }
]
```

Assembled result after masking:

```
[
  { "portfolio_id": "GLOB_EQ_OPP", "client_name": "[REDACTED]", "account_number":
"[REDACTED]", "lcr": 1.24 }
]
```

Projection errors and user-facing messaging

When the projection layer blocks a metric or dimension, the response to the user or AI agent includes a structured error alongside the partial result (for multi-metric queries where some metrics are accessible):

```
{
  "result": {
    "rows": [ ... ],
    "columns": ["portfolio_return", "tracking_error"]
  },
  "projection_errors": [
    {
      "type":       "METRIC_NOT_ENTITLED",
      "metric_id":  "var_95",
      "role":       "portfolio_manager",
      "message":    "The metric 'VaR 95%' is not available to your role. Contact your
Application Admin to request access."
    }
  ]
}
```

The Narrative Synthesis Engine is aware of projection errors and will include an acknowledgement in the narrative: *"Note: VaR 95% was requested but is not available to your current role."*

Entitlement audit

Every projection decision is recorded in the lineage store as part of the execution record:

```
{
  "projection_record": {
    "user_roles":       ["portfolio_manager"],
    "requested_metrics": ["portfolio_return", "tracking_error", "var_95"],
    "projected_metrics": ["portfolio_return", "tracking_error"],
    "blocked_metrics":   [{ "id": "var_95", "reason": "METRIC_NOT_ENTITLED" }],
    "row_predicates":    ["portfolio_id IN ('GLOB_EQ_OPP', 'UK_CORE_INC', 'STRAT_BAL')"],
    "column_masks":      [],
    "predicate_sources": { "portfolio_id IN (...)": "jwt_claim:managed_portfolios" }
  }
}
```

This record is available in the lineage inspector and provides evidence for regulatory entitlement audits.

Entitlement cache

Entitlement profiles (merged role sets, resolved predicates) are cached per session to avoid re-computation on every query:

Cache property	Value
Cache key	SHA-256 of (user_id, tenant_id, sorted role claims)
Cache TTL	Until JWT expiry or role claim change detected
Invalidation	On JWT refresh (new JWT is re-evaluated); on entitlement config change (via Admin API signal)

Cache property	Value
Scope	In-memory per platform edge function instance

If a user's JWT is refreshed with changed role claims during a session, the entitlement cache is invalidated and the projection is re-evaluated on the next query. Results from before the role change are not retroactively restricted.

10 — Content Rendering

Overview

The content rendering layer receives the assembled analytical result, applies the Visualisation Ontology, and produces rendered output — charts, tables, and narrative prose — for display in the `<ai-analytics>` web component. Every output type has a governed rendering path; no output is generated without a registered contract or rule.

Rendering decision rules

The rendering engine evaluates each analytical result in priority order. **The first matching rule wins.**

Priority	Trigger	Rendered as
1	Custom host renderer registered and ontology selects it	**Custom host renderer** — host-provided ES module
2	Intent pattern is `ATTRIBUTION` and waterfall contract matches	**Waterfall chart** — attribution decomposition
3	Intent pattern is `THRESHOLD` and heatmap contract matches	**Heatmap** — threshold matrix
4	Intent pattern is `COMPOSITION` or `DISTRIBUTION` and treemap matches	**Treemap** — part-to-whole
5	Intent pattern is `RELATIONSHIP` and exactly 2 numeric metrics	**Scatter plot** — risk-return or metric correlation
6	Intent pattern is `TREND` and temporal dimension present	**Line chart** — time series
7	Intent pattern is `COMPARISON` and cardinality ≤ 50	**Bar chart** — multi-series comparison
8	Narrative synthesis requested and result rows ≤ threshold	**Narrative card** — governed prose anchored to result
9	All other results	**Governed data table** — sortable, filterable, exportable

---

Rendering output types

Chart rendering

Charts are rendered using the library specified in the tenant's `visualisation.defaultChartLibrary` config. The platform supports three libraries:

Library	Format	Strengths	Host configuration
Vega-Lite	JSON spec	Declarative, composable, excellent for grammar-of-graphics patterns	`"vega-lite"`
Plotly	JSON spec	Financial chart types (candlestick, OHLC), 3D support	`"plotly"`
ECharts	JSON spec	High-performance for large datasets, heatmaps, treemaps	`"echarts"`

All three libraries render from a JSON specification produced by the rendering pipeline. The pipeline populates the specification template from the matching Visualisation Ontology contract and the result schema.

Governed data table

Every chart rendering also produces a **secondary data table** view accessible via a *"Show as table"* toggle. This ensures analytical data is always accessible in raw tabular form regardless of chart type.

Table feature	Specification
Column headers	SMR metric labels and dimension labels — never physical column names
Number formatting	From metric `display.format` and `display.decimals` in the SMR definition
Sorting	Click column header — ascending/descending
Filtering	Inline per-column filter
Pagination	25 rows per page; configurable up to 1,000
Export	CSV and JSON — exports include lineage reference
Sparklines	Inline sparklines for temporal metrics (when a date dimension is present alongside other dimensions)
Threshold highlighting	Cells where a metric exceeds a registered threshold are highlighted per the `sign_convention` in the SMR definition

Narrative card

Narrative synthesis produces a structured prose card composed of:

1. **Lead sentence** — the primary finding: *"Across 14 portfolios, 9 outperformed their benchmark this quarter."*
2. **Supporting detail** — the two or three most analytically significant observations from the result set
3. **Notable outliers** — dimension values at the extreme ends of the metric distribution
4. **Data provenance** — *"Data as of 14 May 2026, Q2 2026 QTD."*
5. **Anchoring disclosure** — a collapsed inspector showing the source result values the narrative was derived from

Narrative synthesis constraints:

Constraint	Enforcement mechanism
No metric values not in the result	Post-generation validation against result set — regenerate if violated
No LLM training data figures	Prompt-level constraint: *"You may only reference values from the provided result set."*
No hedging language about data accuracy	Narrative anchored to governance metadata (data refresh cadence, SLA) in the SMR
No recommendation or investment advice	Prompt-level constraint — describe, do not recommend
Unit-correct formatting	Numbers formatted using SMR metric `display.format` before injection into narrative prompt

Lineage inspector

The lineage inspector is a collapsible disclosure available on every analytical result, accessible to Power Analysts and above:

```
┌──────────────────────────────────────────────────┐
│ 🔍 Query Lineage  [Expand ▼]                     │
└──────────────────────────────────────────────────┘
```

On expansion:

```
┌──────────────────────────────────────────────────┐
│ 🔍 Query Lineage  [Collapse ▲]                   │
├──────────────────────────────────────────────────┤
│ Your question                                    │
│ "Show portfolio returns vs benchmark for Q2 2026"│
├──────────────────────────────────────────────────┤
│ Resolved metrics                                 │
│  • portfolio_return v2.1.0  — Performance › Portfolio
│    Owner: Head of Performance Analytics          │
│  • benchmark_return v1.4.0  — Performance › Benchmark
│    Owner: Head of Performance Analytics          │
├──────────────────────────────────────────────────┤
│ Applied filters                                  │
│  • Row: portfolio_id IN ('GLOB_EQ_OPP', ...)  [Role filter]
├──────────────────────────────────────────────────┤
│ Execution                                        │
│  • Engine: snowflake-primary  (1,240 ms)         │
│  • Engine: dbt-semantic  (890 ms)                │
│  • Total: 1,285 ms  •  Cost: 500 units           │
├──────────────────────────────────────────────────┤
│ Data currency                                    │
│  • portfolio_return: Daily refresh, data as of T+1
│  • benchmark_return: Daily refresh, data as of T+1
├──────────────────────────────────────────────────┤
│ Lineage ID: lqp_20260514_093247_a1b2c3  [Copy]   │
└──────────────────────────────────────────────────┘
```

Streaming behaviour

Content type	Streaming behaviour
Narrative prose	Streams token-by-token; renders incrementally
Chart rendering	Rendered after FQP result assembly completes — not streamed
Data table	Rendered after FQP result assembly completes — not streamed
Lineage inspector	Rendered after lineage record is written — not streamed
Governance warnings	Displayed immediately when triggered (before execution)

While the FQP is executing, the rendering layer displays a **progress indicator** with live status:

```
Resolving metrics…   ✓
Applying entitlements…   ✓
Planning query…   ✓
Executing (2 engines)…   ⏳ 890 ms
```

Analytical result artefact

Every analytical result produces an **artefact** stored in the session artefact tray:

Artefact element	Content
Chart image	SVG export of the rendered chart
Data table	CSV of the underlying result set
Lineage document	JSON lineage record
Narrative text	Plain text of the narrative synthesis
DSL expression	The internal DSL expression that produced the result

Artefacts are downloadable from the tray individually or as a zip archive. Exports are governed by the `requireLineageForExport` setting — if enabled, the lineage document is automatically bundled with every export.

Custom host renderers

Host applications may register custom rendering modules for domain-specific visualisations not covered by the standard chart contracts:

```
{
  "id":       "risk-gauge",
  "trigger":  "risk-gauge",
  "name":     "Risk Gauge",
  "moduleUrl": "https://cdn.acme.com/analytics-renderers/risk-gauge.js",
  "systemPromptGuidance": "Use the risk-gauge renderer when presenting a single VaR
utilisation figure as a dial. The result must contain exactly: { score: 0–100, label: string,
threshold: number }."
}
```

Custom renderers receive the result set and the selected chart contract from the Visualisation Ontology. They are loaded as ES modules and rendered in an isolated shadow DOM, following the same renderer contract as the AI Chat Platform.

Error states in rendering

Error	User-facing message	Rendering fallback
No matching chart contract	*"This result cannot be represented as a chart — showing as table."*	Governed data table
Chart library render failure	*"Chart rendering failed — showing raw data."*	Governed data table
Custom renderer load failure	*"[Renderer name] could not load — showing standard chart."*	Standard ontology chart
Result cardinality exceeds threshold	*"Result has N rows — chart limited to first 10,000. Download the full dataset below."*	Chart with sampled data + full CSV download
Narrative synthesis constraint violation	*(Narrative regenerated internally — user sees second-attempt narrative or, if two attempts fail, no narrative is shown)*	Chart + table only; no narrative

11 — Audit and Storage

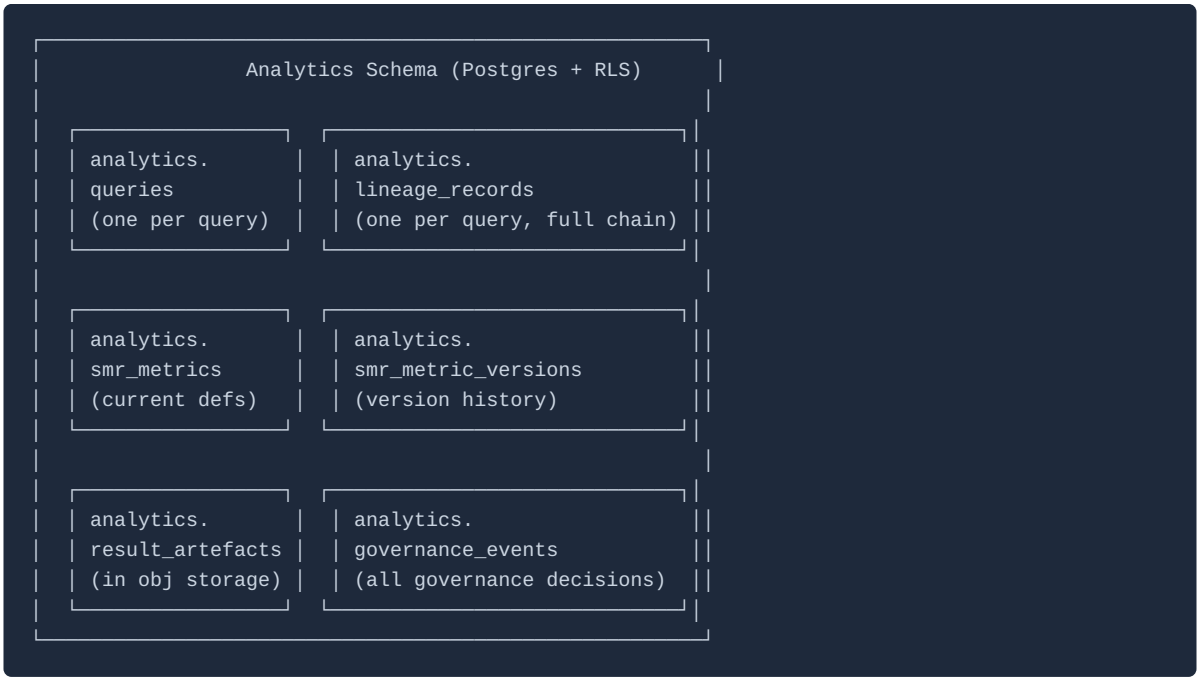
Governing principle

Every analytical query on the AI Analytics Platform produces a complete, queryable lineage record. The audit trail is not a log — it is a first-class data structure that supports regulatory enquiry, governance review, and analytical reproducibility. A regulator, auditor, or internal reviewer must be able to reconstruct exactly what was queried, by whom, under what entitlements, using which metric definitions, through which execution engines, and with what results — without reference to any external system.

Multi-tenant isolation

Every record in the `analytics` schema carries a `tenant_id` column. Row-Level Security (RLS) enforces that users can only access records belonging to their own tenant. No cross-tenant data access is possible through the platform's API.

Storage architecture



Per-query stored elements

Element	Table	Content
Query record	`analytics.queries`	User ID, tenant ID, raw natural language, DSL expression, intent pattern, timestamp, governance status, execution status, cost units consumed
Lineage record	`analytics.lineage_records`	Complete chain: intent → SMR resolution → projection record → LQP → FQP execution record → result schema → visualisation contract → narrative synthesis status
SMR snapshot	`analytics.lineage_records.metric_versions`	For each metric in the query: metric ID, SMR definition version, formula at query time
Projection record	`analytics.lineage_records.projection_record`	Roles, requested metrics, projected metrics, blocked metrics, row predicates, column masks
FQP execution record	`analytics.lineage_records.fqp_execution`	Sub-plan details, engine IDs, latencies, cost units, cache hit status
Governance decision	`analytics.governance_events`	Circuit breaker decisions, classification gates, cost limit checks — including blocked queries
Result artefact	Object storage + `analytics.result_artefacts`	CSV result set, chart SVG, narrative text — stored per query

Database schema overview

`analytics.queries`

```
CREATE TABLE analytics.queries (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id           TEXT NOT NULL,
  user_id             TEXT NOT NULL,
  session_id          TEXT NOT NULL,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  natural_language    TEXT,
  dsl_expression      TEXT,
  intent_pattern      TEXT,
  governance_status   TEXT NOT NULL,  -- 'approved', 'blocked', 'partial'
  execution_status    TEXT NOT NULL,  -- 'success', 'error', 'timeout', 'cancelled'
  cost_units          INTEGER,
  total_latency_ms    INTEGER,
  result_rows         INTEGER,
  cache_hit           BOOLEAN,
  lineage_id          UUID REFERENCES analytics.lineage_records(id)
);
```

`analytics.lineage_records`

```
CREATE TABLE analytics.lineage_records (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  query_id            UUID NOT NULL REFERENCES analytics.queries(id),
  tenant_id           TEXT NOT NULL,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  -- Structured JSONB fields for each lineage chain element
  intent_resolution   JSONB NOT NULL,  -- natural language → intent mapping
  smr_resolution      JSONB NOT NULL,  -- metric/dimension definitions used
  projection_record   JSONB NOT NULL,  -- entitlement decisions
  lqp                 JSONB NOT NULL,  -- Logical Query Plan
  governance_checks   JSONB NOT NULL,  -- governance decisions
  fqp_execution       JSONB NOT NULL,  -- FQP sub-plans and engine calls
  visualisation       JSONB,           -- chart contract selected
  narrative_status    JSONB            -- narrative synthesis outcome
);
```

`analytics.governance_events`

```
CREATE TABLE analytics.governance_events (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  query_id            UUID REFERENCES analytics.queries(id),
  tenant_id           TEXT NOT NULL,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  event_type          TEXT NOT NULL,   -- 'cost_limit_check', 'classification_gate',
'complexity_limit', 'circuit_breaker'
  decision            TEXT NOT NULL,   -- 'approved', 'blocked', 'warned'
  cost_estimate       INTEGER,
  cost_limit          INTEGER,
  classification      TEXT,
  blocked_classification TEXT,
  reason              TEXT
);
```

`analytics.smr_metrics`

```
CREATE TABLE analytics.smr_metrics (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id           TEXT NOT NULL,
  metric_id           TEXT NOT NULL,
  current_version     TEXT NOT NULL,
  status              TEXT NOT NULL,   -- 'draft', 'proposed', 'approved', 'deprecated',
'retired'
  definition          JSONB NOT NULL,  -- full metric definition YAML, stored as JSONB
  owner               TEXT NOT NULL,
  steward             TEXT,
  approved_by         TEXT,
  approved_at         TIMESTAMPTZ,
  effective_from      DATE,
  deprecated_at       TIMESTAMPTZ,
  UNIQUE(tenant_id, metric_id)
);
```

`analytics.smr_metric_versions`

```
CREATE TABLE analytics.smr_metric_versions (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id           TEXT NOT NULL,
  metric_id           TEXT NOT NULL,
  version             TEXT NOT NULL,
  definition          JSONB NOT NULL,
  proposed_by         TEXT,
  approved_by         TEXT,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  FOREIGN KEY (tenant_id, metric_id) REFERENCES analytics.smr_metrics(tenant_id, metric_id)
);
```

Retention

Rule	Specification
Query records	Configurable per tenant. Platform default: **2,555 days (7 years)** — to cover most regulatory audit look-back periods.
Lineage records	Retained at least as long as the corresponding query record. Cannot be deleted independently.
SMR metric versions	Retained indefinitely — metric version history must be preserved for lineage reconstruction.
Governance events	Retained at least as long as query records.
Result artefacts (object storage)	Configurable per tenant. Default: **365 days**. Lineage record references are preserved even after object storage expiry.
Blocked queries	Retained in full — queries that fail governance checks are as important to retain as successful ones.

Access control

Access level	Capability
Authenticated end user	Read access to their own query records and lineage records within their tenant only
Power Analyst	Read access to lineage inspector for their own queries
Application Admin	Read access to all query records, lineage records, and governance events within their own tenant
Metric Owner	Read access to all queries that used their owned metrics (via SMR audit view)
Platform Admin	Read access to all records across all tenants for operational purposes
No user	May modify or delete query records or lineage records — all records are immutable after writing

Regulatory audit support

The platform supports regulatory audit requests via the Admin API:

```
GET /v1/audit/queries
    ?user_id={user_id}
    &from={ISO8601}
    &to={ISO8601}
    &metric_ids[]={metric_id}
    &include_lineage=true
    → Returns all queries matching the filter with full lineage records
```

Audit export packages (for submission to regulatory bodies) include: - Query records with timestamps and user identifiers - Lineage records with metric definition versions used - Governance decisions for each query - Role-aware projection records showing entitlements in force at query time - Result artefacts (if within retention period)

All export packages are digitally signed by the platform using a tenant-specific key registered at onboarding.

12 — Governed Execution

Overview

The **Semantic Execution Governance** layer applies a suite of circuit breakers, cost controls, complexity limits, and compliance classification checks to every analytical query before it is released to the Federated Query Planner. It is the last gate before physical execution.

Governance is applied to every query without exception. There is no privileged user, trusted agent, or internal bypass path that skips governance checks.

Governance pipeline

```
Validated LQP (post role-aware projection)
│
├── 1. Cost estimation
│       Estimate execution cost units from LQP metadata
│       (cardinality estimate × engine cost tier × complexity factor)
│
├── 2. Cost circuit breaker
│       Compare estimated cost to maxQueryCostUnits
│       BLOCK if exceeded → user prompted to narrow scope
│
├── 3. Complexity limit check
│       Evaluate LQP node count, join depth, sub-plan count
│       BLOCK if exceeds complexity threshold
│
├── 4. Classification gate
│       For each metric: retrieve data.classification from SMR
│       Compare against blockedClassifications list
│       BLOCK if any metric's classification is in blocked list
│
├── 5. Regulatory compliance mode check
│       If complianceMode is set: apply compliance-specific rules
│       (e.g. MiFID II: log all queries involving client-related metrics)
│
├── 6. Concurrency limit check
│       Count active queries for this user
│       BLOCK (with wait) if exceeds maxConcurrentQueries
│
├── 7. Timeout budget assignment
│       Assign queryTimeoutSeconds to the FQP execution context
│
└── 8. Governance approval record written
        Governance event record written before FQP is invoked
        (ensures governance decisions are auditable even if FQP fails)
        → Release to FQP
```

Cost estimation model

Cost units are estimated from the LQP before execution. The estimation model uses:

Factor	Contribution
Number of metrics	`metric_count × 50` base units
Engine cost tier per sub-plan	`minimal: 10`, `low: 50`, `standard: 100`, `high: 300`, `unrestricted: 0`
Dimension cardinality	`low: ×1.0`, `medium: ×1.5`, `high: ×3.0`, `unbounded: ×5.0`
Time period scope	`single_day: ×1.0`, `quarter: ×2.0`, `year: ×4.0`, `since_inception: ×8.0`
Number of sub-plans (federation)	`+100 per additional sub-plan`
Materialised view match	`-800` (pre-computed result)
Cache hit (estimated)	`-900` (full cache hit expected)

Example cost estimate:

Query: `portfolio_return, tracking_error BY portfolio, asset_class FOR YEAR_TO_DATE`

```
portfolio_return:   50 (metric base)
tracking_error:     50 (metric base)
snowflake engine:  100 (standard cost tier)
dbt engine:         50 (low cost tier)
asset_class:       1.5× cardinality multiplier (medium)
YEAR_TO_DATE:      4.0× period multiplier
2 sub-plans:       100 (federation overhead)
──────────────────────────────
Base: (50+50) × 1.5 × 4.0 = 600
Engines: 100+50 = 150
Federation: 100
Total estimate: 850 cost units
```

Against a `maxQueryCostUnits: 1000` limit, this query is approved. Against a `500` limit, it is blocked.

Cost circuit breaker response

When a query is blocked by the cost circuit breaker, the user receives a structured response:

```
┌──────────────────────────────────────────────────────┐
│ ⚠️  Query scope is too broad                          │
├──────────────────────────────────────────────────────┤
│ Estimated cost: 850 units (limit: 500 units)         │
│                                                      │
│ Suggestions to reduce scope:                         │
│  • Narrow the time period (e.g. use Quarter-to-Date  │
│    instead of Year-to-Date)                          │
│  • Reduce the number of metrics (e.g. query 'portfolio_ │
│    return' separately from 'tracking_error')         │
│  • Filter to a specific portfolio or asset class     │
│                                                      │
│ [Refine my question]    [Contact Application Admin]  │
└──────────────────────────────────────────────────────┘
```

The AI assistant reformulates the query based on the suggestions if the user clicks *"Refine my question"*.

Classification gate

The classification gate evaluates the data classification of every metric in the query against the tenant's `blockedClassifications` list.

Classification evaluation logic:

```
def classification_gate(metrics, smr, blocked_classifications):
    for metric in metrics:
        smr_def = smr.get(metric.id)
        if smr_def.governance.classification in blocked_classifications:
            raise GovernanceBlock(
                type="CLASSIFICATION_GATE",
                metric_id=metric.id,
                classification=smr_def.governance.classification,
                message=f"Metric '{smr_def.label}' has classification "
                        f"'{smr_def.governance.classification}' which is "
                        f"not permitted for query execution."
            )
    return "approved"
```

Classification gate blocks are non-negotiable — they cannot be overridden by the user or Application Admin within the session. Changing the classification gate configuration requires an Admin API config update.

Compliance modes

Named compliance profiles pre-configure governance behaviour for specific regulatory environments:

MiFID II mode

```
{ "complianceMode": "mifid2" }
```

Additional rule	Implementation
All queries involving client-identifiable data must be logged with the business justification	Prompt user for business justification before queries on `client_name`, `account_number`, or similar PII-adjacent dimensions
Best execution metrics must be queried with explicit timeframe	DSL compilation error if `date` dimension not specified for best-execution metrics
Transaction reporting queries must generate a TRACE record	Additional lineage record written to `analytics.mifid2_trace` table

Basel III/IV mode

```
{ "complianceMode": "basel3" }
```

Additional rule	Implementation
Capital ratio queries must include entity identifier	Required dimension: `entity` for all regulatory capital metrics
LCR/NSFR queries generate a daily snapshot record	Regulatory metric queries trigger a snapshot write to `analytics.regulatory_snapshots`
Queries on stress scenario data classified as RESTRICTED	Stress scenario metrics automatically classified at RESTRICTED level regardless of user role

SEC Regulation BI mode

```
{ "complianceMode": "sec_reg_bi" }
```

Additional rule	Implementation
Narrative synthesis prohibited from generating investment recommendations	Additional narrative synthesis constraint injected into prompt
Client analytics require suitability record reference	Advisory queries require `suitability_record_id` parameter before execution

---

Timeout and partial result handling

The FQP applies the `queryTimeoutSeconds` budget assigned by the governance layer. Timeout behaviour:

Scenario	Behaviour
All sub-plans complete within timeout	Normal result assembly and return

Scenario	Behaviour
One sub-plan times out, others complete	Partial result assembly — missing metrics represented as null with `timeout` provenance marker; user notified
All sub-plans time out	Query failed — error returned to user; governance event written with `timeout` status
Engine cancellation on timeout	FQP sends cancellation signal to timed-out engine (if engine supports cancellation)

Governance event record

Every governance evaluation — approved or blocked — produces a governance event record:

```
{
  "id":             "gov_evt_20260514_093247",
  "query_id":       "qry_20260514_093247",
  "tenant_id":      "acme-wealth",
  "created_at":     "2026-05-14T09:32:47.123Z",
  "checks": [
    {
      "type":       "cost_estimate",
      "decision":   "approved",
      "estimated": 500,
      "limit":      1000
    },
    {
      "type":       "complexity_limit",
      "decision":   "approved",
      "lqp_nodes": 8,
      "limit":      50
    },
    {
      "type":       "classification_gate",
      "decision":   "approved",
      "metrics_checked": ["portfolio_return", "tracking_error"],
      "highest_classification": "INTERNAL",
      "blocked_classifications": ["TOP_SECRET", "RESTRICTED"]
    },
    {
      "type":       "concurrency_limit",
      "decision":   "approved",
      "active_queries": 1,
      "limit":      5
    }
  ],
  "overall_decision": "approved",
  "fqp_released_at": "2026-05-14T09:32:47.201Z"
}
```

13 — Financial Services Semantic Model

Overview

The platform ships a **Financial Services Reference Semantic Model** — a pre-built set of metric definitions, dimensions, hierarchies, and measure groups covering the principal analytical domains of wealth management, banking, investment management, and regulatory reporting.

Host applications seeding their SMR from `analyticalDomain: "wealth_management"`, `"banking"`, or `"investment_management"` receive the relevant subset of this reference model as their baseline. Definitions may be customised, extended, or superseded by tenant-specific definitions via the Admin API.

Analytical domains

Domain ID	Description	Applicable to
`portfolio`	Portfolio structure, positions, market values, cash flows	Wealth management, investment management
`performance`	Return metrics, attribution, benchmark comparison	Wealth management, investment management
`risk`	Market risk, credit risk, liquidity risk, VaR, factor exposures	All investment entities
`regulatory`	Capital adequacy, liquidity ratios, leverage ratios, reporting metrics	Banking, regulated investment firms
`counterparty`	Counterparty exposure, settlement risk, credit exposure	Banking, institutional trading
`benchmarks`	Index and benchmark data, benchmark decomposition	Wealth management, investment management

---

Portfolio domain

Dimensions

Dimension ID	Label	Type	Cardinality	Description
`portfolio`	Portfolio	categorical	medium	Portfolio identifier
`asset_class`	Asset Class	categorical	low	Broad asset type (Equity, Fixed Income, Alternatives, Cash)

Dimension ID	Label	Type	Cardinality	Description
`sub_asset_class`	Sub-Asset Class	categorical	medium	Granular asset type
`security_type`	Security Type	categorical	medium	Instrument type (Common Stock, Corporate Bond, etc.)
`issuer`	Issuer	categorical	high	Issuer of the security
`geography`	Geography	categorical	medium	Country or region of domicile
`currency`	Currency	categorical	low	Instrument currency
`rating`	Credit Rating	categorical	low	External credit rating (AAA through D)
`date`	Date	temporal	unbounded	Valuation date
`sector`	Sector	categorical	medium	GICS or ICB sector
`benchmark`	Benchmark	categorical	low	Associated benchmark identifier

Core portfolio metrics

```
# AUM — Assets Under Management
metric:
  id:          "aum"
  label:       "Assets Under Management"
  description: "Total market value of assets managed in the portfolio, expressed in the
portfolio's base currency."
  formula:     "SUM(position_market_value)"
  unit:        "currency"
  aggregation:
    default:   "sum"
    allowed:   ["sum"]
    granularity: ["daily", "monthly", "quarterly", "annual"]
  dimensions:
    required:  ["portfolio", "date"]
    optional:  ["asset_class", "currency", "geography"]
  data:
    domain:    "portfolio"
    refresh_cadence: "daily"
  governance:
    classification: "INTERNAL"
```

```
# Issuer Concentration
metric:
  id:          "issuer_concentration"
  label:       "Issuer Concentration"
  description: "Market value of positions in a given issuer as a percentage of total portfolio
AUM."
  formula:     "SAFE_DIVIDE(SUM(position_market_value, FILTER(issuer = {{dim.issuer}})),
SUM(position_market_value))"
  unit:        "percentage"
  aggregation:
    default:   "value_weighted_average"
    allowed:   ["value_weighted_average", "sum"]
    granularity: ["daily"]
  dimensions:
    required:  ["portfolio", "issuer", "date"]
  governance:
    classification: "INTERNAL"
```

Performance domain

Core performance metrics

```
# Portfolio Return (Total Return, Net of Fees)
metric:
  id:          "portfolio_return"
  version:     "2.1.0"
  label:       "Portfolio Return"
  description: "Total return of the portfolio over the specified period, net of management and
performance fees, expressed as a percentage. Calculated using the Modified Dietz method for
sub-period returns, chain-linked for multi-period returns."
  formula:     "CHAIN_LINK(modified_dietz_return, time_period)"
  unit:        "percentage"
  aggregation:
    default:   "value_weighted_average"
    allowed:   ["value_weighted_average", "equal_weighted_average"]
    granularity: ["daily", "monthly", "quarterly", "annual", "since_inception"]
  display:
    format:    "percentage"
    decimals:  2
    benchmark_comparison: true

metric:
  id:          "benchmark_return"
  label:       "Benchmark Return"
  description: "Total return of the portfolio's designated benchmark index over the same
period."
  formula:     "CHAIN_LINK(index_daily_return, time_period)"
  unit:        "percentage"
  dimensions:
    required:  ["portfolio", "benchmark", "date"]

metric:
  id:          "active_return"
  label:       "Active Return"
  description: "Portfolio return minus benchmark return (alpha). Positive active return
indicates outperformance."
  formula:     "portfolio_return - benchmark_return"
  unit:        "percentage"
  lineage:
    upstream_metrics: ["portfolio_return", "benchmark_return"]

metric:
  id:          "tracking_error"
  label:       "Tracking Error"
  description: "Annualised standard deviation of the difference between portfolio returns and
benchmark returns. Measures consistency of active return delivery."
  formula:     "ANNUALISE(STDDEV(portfolio_return - benchmark_return, time_period))"
  unit:        "percentage"
  aggregation:
    granularity: ["monthly", "quarterly", "annual"]

metric:
  id:          "information_ratio"
  label:       "Information Ratio"
  description: "Active return divided by tracking error. Measures risk-adjusted active return
efficiency. Values above 0.5 indicate strong risk-adjusted outperformance."
  formula:     "SAFE_DIVIDE(active_return, tracking_error)"
  unit:        "ratio"
  display:
    decimals:  2
    sign_convention: "positive_is_good"

metric:
  id:          "sharpe_ratio"
```

```
  label:       "Sharpe Ratio"
  description: "Portfolio excess return above the risk-free rate divided by portfolio
volatility. Measures risk-adjusted absolute return."
  formula:     "SAFE_DIVIDE(portfolio_return - risk_free_rate, portfolio_volatility)"
  unit:        "ratio"
```

Risk domain

Market risk metrics

```
metric:
  id:          "var_95"
  label:       "Value at Risk (95%)"
  description: "The maximum expected loss over a 1-day horizon at a 95% confidence level,
expressed as a percentage of portfolio AUM. Calculated using historical simulation over a 250-
day lookback."
  formula:     "PERCENTILE(portfolio_daily_pnl_pct, 0.05) × -1"
  unit:        "percentage"
  aggregation:
    granularity: ["daily"]
  governance:
    classification: "INTERNAL"
    owner:       "head_of_risk"

metric:
  id:          "var_99"
  label:       "Value at Risk (99%)"
  description: "Maximum expected loss over a 1-day horizon at a 99% confidence level."
  formula:     "PERCENTILE(portfolio_daily_pnl_pct, 0.01) × -1"
  unit:        "percentage"

metric:
  id:          "cvar"
  label:       "Conditional Value at Risk (CVaR)"
  description: "Expected loss given that the loss exceeds the 95% VaR threshold. Also known as
Expected Shortfall."
  formula:     "MEAN(portfolio_daily_pnl_pct, FILTER(portfolio_daily_pnl_pct <
PERCENTILE(portfolio_daily_pnl_pct, 0.05))) × -1"
  unit:        "percentage"

metric:
  id:          "beta"
  label:       "Portfolio Beta"
  description: "Systematic risk of the portfolio relative to the benchmark. Beta of 1.0 means
the portfolio moves in line with the benchmark."
  formula:     "COVARIANCE(portfolio_return, benchmark_return) / VARIANCE(benchmark_return)"
  unit:        "ratio"

metric:
  id:          "duration"
  label:       "Portfolio Duration"
  description: "Market-value-weighted average modified duration of the fixed income portfolio
in years."
  formula:     "SAFE_DIVIDE(SUM(position_market_value × security_modified_duration),
SUM(position_market_value))"
  unit:        "years"

metric:
  id:          "credit_spread_dv01"
  label:       "Credit Spread DV01"
  description: "Change in portfolio value for a 1 basis point parallel shift in credit
spreads, expressed in base currency."
  formula:     "SUM(position_market_value × security_cs01)"
  unit:        "currency"
```

Regulatory domain

Liquidity metrics (Banking)

```
metric:
  id:          "lcr"
  label:       "Liquidity Coverage Ratio"
  description: "High Quality Liquid Assets (HQLA) as a proportion of Net Cash Outflows (NCO)
over a 30-day stress period. Basel III minimum: 100%. Expressed as a percentage."
  formula:     "SAFE_DIVIDE(hqla_total, net_cash_outflows_30d) × 100"
  unit:        "percentage"
  governance:
    classification:    "RESTRICTED"
    owner:             "treasury_risk_management"
  display:
    decimals:          1
    benchmark_comparison: false

metric:
  id:          "nsfr"
  label:       "Net Stable Funding Ratio"
  description: "Available Stable Funding (ASF) as a proportion of Required Stable Funding
(RSF). Basel III minimum: 100%."
  formula:     "SAFE_DIVIDE(available_stable_funding, required_stable_funding) × 100"
  unit:        "percentage"

metric:
  id:          "leverage_ratio"
  label:       "Leverage Ratio"
  description: "Tier 1 Capital divided by Total Exposure. Basel III minimum: 3%."
  formula:     "SAFE_DIVIDE(tier1_capital, total_exposure) × 100"
  unit:        "percentage"
```

Hierarchies reference

Asset class hierarchy

```
Asset Class (Level 1)
├── Equity
│   ├── Developed Market Equity
│   │   ├── Large Cap
│   │   ├── Mid Cap
│   │   └── Small Cap
│   └── Emerging Market Equity
├── Fixed Income
│   ├── Government Bonds
│   │   ├── Developed Market Government
│   │   └── Emerging Market Government
│   ├── Corporate Bonds
│   │   ├── Investment Grade
│   │   └── High Yield
│   └── Securitised
├── Alternatives
│   ├── Private Equity
│   ├── Real Estate
│   ├── Infrastructure
│   └── Hedge Funds
└── Cash & Money Market
```

Geography hierarchy

```
Geography (Level 1: Region)
├── Europe, Middle East & Africa
│   ├── Western Europe
│   │   └── [Country level]
│   ├── Eastern Europe
│   └── Middle East & Africa
├── Americas
│   ├── North America
│   ├── Latin America
│   └── Caribbean
└── Asia Pacific
    ├── Developed Asia
    ├── Emerging Asia
    └── Australasia
```

Time hierarchy

```
Year (Level 1)
└── Quarter
    └── Month
        └── Week
            └── Day
```

Measure groups reference

Measure Group ID	Label	Metrics included
`performance_metrics`	Performance Metrics	`portfolio_return`, `benchmark_return`, `active_return`, `tracking_error`, `information_ratio`, `sharpe_ratio`
`risk_metrics`	Risk Metrics	`var_95`, `var_99`, `cvar`, `beta`, `duration`, `credit_spread_dv01`
`portfolio_summary`	Portfolio Summary	`aum`, `portfolio_return`, `benchmark_return`, `active_return`, `var_95`
`concentration_metrics`	Concentration Metrics	`issuer_concentration`, `asset_class_weight`, `geography_weight`, `sector_weight`
`regulatory_metrics`	Regulatory Metrics	`lcr`, `nsfr`, `leverage_ratio`

14 — Success Metrics

Governing principle

Metrics are captured from day one at both the platform level (across all tenants) and at the application level (per tenant). Governance health metrics are treated as first-class success indicators — not vanity metrics. A platform that is highly used but poorly governed is not successful.

Platform-level metrics

Metric	Definition	Target
Active tenants	Tenants with at least one successful query execution in the 7-day window	Growth metric — tracked weekly
Platform uptime	API availability (p99 latency < 2s; error rate < 0.1%)	99.9%
Governance block rate	Queries blocked by governance checks ÷ total queries	Monitor — sustained > 30% may indicate misconfigured cost limits or entitlements
FQP error rate	Queries with FQP execution errors ÷ total executed queries	< 2%
Cache hit rate	Queries served from result cache ÷ total executed queries	≥ 35% by month 2
Lineage completeness	Queries with complete lineage records ÷ total queries	100% — any deviation is a platform defect
SMR registry health	Metrics with no owner OR not updated in 12 months ÷ total active metrics	< 5%

Application-level metrics

Usage metrics

Metric	Definition	Target
Weekly active users (WAU)	Distinct users executing at least one query in a 7-day window	50% of entitled users by day 90
Queries per user per week	Total query executions ÷ weekly active users	≥ 8

Metric	Definition	Target
Drilldown rate	Sessions with at least one drilldown traversal ÷ total sessions	≥ 20%
Export rate	Sessions with at least one result export ÷ total sessions	≥ 15%
Narrative synthesis consumption	Queries where the user expanded the narrative card ÷ total queries with narrative	≥ 40%
Lineage inspector opens	Queries where user opened the lineage inspector ÷ total queries (Power Analysts)	≥ 25%

Governance metrics

Metric	Definition	Target
Metric resolution success rate	Queries where all requested metrics resolved from SMR ÷ total queries	≥ 95%
Cost circuit breaker rate	Queries blocked by cost limits ÷ total queries	< 5% — sustained > 10% triggers review of cost limit configuration
Classification gate block rate	Queries blocked by classification gate ÷ total queries	Monitored — any unexpected spike triggers security review
Entitlement error rate	Queries with METRIC_NOT_ENTITLED or DIMENSION_NOT_ENTITLED errors ÷ total queries	< 3%
SMR approval backlog	Metric definitions in `proposed` or `in_review` state > 7 days	0 — all pending definitions reviewed within 7 days
Metric owner coverage	Active metrics with an assigned owner ÷ total active metrics	100%

Data quality metrics

Metric	Definition	Target
Execution engine error rate	Engine sub-plan failures ÷ total sub-plan executions (per engine)	< 1% per engine
Partial result rate	Queries returning partial results (timeout on one or more sub-plans) ÷ total queries	< 2%
Stale data indicator rate	Queries where a metric's data refresh cadence exceeded its SLA ÷ total queries	< 5%
Cache freshness	Cached results served beyond their TTL (due to cache invalidation delay) ÷ cache hits	< 1%

Query quality metrics

Metric	Definition	Target
Intent resolution accuracy	Queries where the user accepted the resolved intent without modification (intent confirmation enabled tenants) ÷ total queries	≥ 85%
Query reformulation rate	Queries where the user rephrased within 2 turns after a resolution error ÷ total queries	< 10%
Narrative validation failure rate	Narrative synthesis attempts failing post-generation validation ÷ total narrative syntheses	< 2%

Metric definitions

WAU measurement

A user is counted once per 7-day window regardless of query count. "Active" means at least one successfully executed query (governance-approved + FQP-executed). Queries blocked at governance are not counted.

Governance block rate interpretation

Block rate	Interpretation
< 5%	Healthy — some queries are appropriately blocked
5–15%	Review recommended — entitlement or scope configuration may need tuning
15–30%	Likely misconfiguration — cost limits, entitlements, or scope too restrictive
> 30%	Configuration review mandatory — platform may be inaccessible to legitimate queries

Lineage completeness

Measured as `completed_lineage ÷ total_queries`. Any value below 100% triggers an automated platform alert. Lineage gaps are platform defects, not acceptable operational variance.

SMR approval backlog

Measured as the count of metric definitions that have been in `proposed` or `in_review` state for more than 7 calendar days. A non-zero count triggers a notification to the Application Admin.

Review cadence

Cadence	Activity	Owner
Daily	Lineage completeness check; FQP error rate; classification gate spike detection	Platform Engineering (automated)
Weekly	WAU; governance block rate; SMR approval backlog; engine error rate per tenant	Platform Engineering + Application Admin
Monthly	Full metric review; query quality analysis; SMR health report; narrative validation rate	Platform team + Application Admins
Day 90 (per tenant)	WAU adoption target assessment (50%); drilldown adoption; export rate	Application Admin + Platform team
Quarterly	SMR completeness review; metric owner coverage; entitlement policy review	Application Admin + Metric Owners

---

Tenant analytics dashboard

Each tenant's Application Admin has access to a read-only analytics dashboard showing:

- WAU trend (30-day rolling)
- Query volume by intent pattern
- Governance block rate trend with breakdown by circuit breaker type
- SMR metric resolution success rate
- Execution engine error rate per engine
- Cache hit rate trend
- Top 10 most-queried metrics
- Lineage completeness (always 100% or an active alert)
- SMR approval backlog count

Platform-level cross-tenant metrics are visible to the Platform team only.

15 — Embedding and Web Component

Overview

The `<ai-analytics>` component is a **custom HTML element** that host applications embed within their own UI. It is self-contained — it manages its own state, handles the full analytical pipeline from natural language through to rendered results, and communicates with the platform API independently of the host application's framework.

The component delivers a three-zone layout:

Zone	Location	Contents
Query history panel	Left sidebar	Session query history, SMR browser, saved queries
Analytical workspace	Centre — primary	Query input, rendered results, narrative, lineage inspector
Context panel	Right sidebar	Active metric definitions, result artefacts, drilldown breadcrumb, session summary

Quick start

```
<script type="module" src="https://analytics-platform.io/sdk/v1/ai-analytics.js"></script>

<ai-analytics
  tenant-id="acme-wealth"
  user-token="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  theme="light"
></ai-analytics>
```

The component renders at 100% of its container's width and height.

HTML attributes

Attribute	Required	Type	Description
`tenant-id`	Yes	string	Tenant identifier from the application config
`user-token`	Yes	string	Host-authenticated user JWT. Must contain role claims for projection.
`theme`	No	`"light"` | `"dark"`	Colour scheme. Default: `"light"`.

Attribute	Required	Type	Description
`locale`	No	string	BCP 47 locale tag. Overrides tenant config `scope.language` for UI strings and number formatting.
`initial-query`	No	string	Pre-populate the query input with a specific question on mount. Useful for deep-linking from the host application.
`smr-panel-visible`	No	boolean	Whether the SMR browser panel opens on first mount. Default: `false`.

Authentication bridge

The platform validates the host-issued JWT and extracts:

Required JWT claims

Claim	Type	Description
`sub`	string	User's unique identifier within the tenant
`tenant_id`	string	Must match the `tenant-id` attribute
`exp`	number	JWT expiry time

Required analytical claims

Claim	Type	Description
Any field matching `entitlements.roleClaimField`	string[]	User's analytical role array. Used by the Role-Aware Projection Layer.

Optional claims

Claim	Type	Description
`managed_portfolios`	string[]	Portfolio IDs managed by this user — resolved in row predicate templates
`entity_ids`	string[]	Legal entities the user is associated with — for multi-entity deployments
`display_name`	string	User's display name for lineage records and audit trail
Any other claim referenced in role `rowPredicates`	*	Any claim value referenced by `{{user.claim_name}}` in predicate templates

---

JavaScript API

```
const analytics = document.querySelector('ai-analytics');
```

Methods

Method	Parameters	Description
`updateToken(token)`	`token: string`	Refresh the user JWT without remounting
`submitQuery(query)`	`query: string`	Programmatically submit a natural language query
`clearSession()`	—	Clear the current session's query history and results
`openSMRBrowser()`	—	Open the SMR metric browser panel
`setLocale(locale)`	`locale: string`	Change number formatting locale at runtime

Properties

Property	Type	Description
`activeMetrics`	`string[]`	Metric IDs in the most recent result
`isExecuting`	`boolean`	Whether a query is currently in execution
`lastLineageId`	`string | null`	Lineage ID of the most recent completed query

Events

Event	Detail payload	Description
`query-submitted`	`{ naturalLanguage, sessionId }`	Fired when user submits a query
`intent-resolved`	`{ intentPattern, metrics, dimensions, dslExpression }`	Fired after Semantic Intent Layer resolution
`governance-blocked`	`{ reason, circuitBreaker, estimatedCost, costLimit }`	Fired when a query is blocked by governance
`query-complete`	`{ lineageId, metrics, rowCount, latencyMs, cacheHit }`	Fired when a query completes successfully
`query-error`	`{ error, errorType, lineageId }`	Fired on execution error
`drilldown-navigated`	`{ hierarchy, level, selectedValue, lineageId }`	Fired on governed drilldown traversal
`result-exported`	`{ format, lineageId, filename }`	Fired when user exports a result

Event	Detail payload	Description
`metric-definition-viewed`	`{ metricId, metricVersion }`	Fired when user views a metric definition from the SMR browser
`token-expired`	—	Fired when user JWT approaches expiry

Handling `governance-blocked`

```
analytics.addEventListener('governance-blocked', (event) => {
  const { reason, circuitBreaker, estimatedCost, costLimit } = event.detail;

  // Host application may log this for operational monitoring
  telemetry.track('analytics.governance_block', {
    circuit_breaker: circuitBreaker,
    estimated_cost:  estimatedCost,
    cost_limit:      costLimit
  });
});
```

Handling `query-complete` for host application integration

```
analytics.addEventListener('query-complete', (event) => {
  const { lineageId, metrics, rowCount } = event.detail;

  // Host application may link analytical results to their own workflow
  if (metrics.includes('var_95') && rowCount > 0) {
    riskManagementApp.flagForReview(lineageId);
  }
});
```

Sizing and layout

The `<ai-analytics>` component renders at **100% width and 100% height** of its container.

Minimum dimensions

Dimension	Minimum
Width	480px
Height	500px

Below minimum dimensions, a fallback state is displayed.

Responsive layout breakpoints

Viewport width	Layout
≥ 1280px	Full three-zone layout
1024px–1279px	Context panel collapses to icon rail
768px–1023px	Both panels as slide-in drawers
< 768px	Single-column; query input pinned to bottom; panels as bottom sheets

---

Content Security Policy

```
script-src    https://analytics-platform.io [renderer module origins];
connect-src   https://api.analytics-platform.io;
img-src       https://cdn.analytics-platform.io;
style-src     https://analytics-platform.io 'nonce-{page-nonce}';
```

Framework wrappers

React:

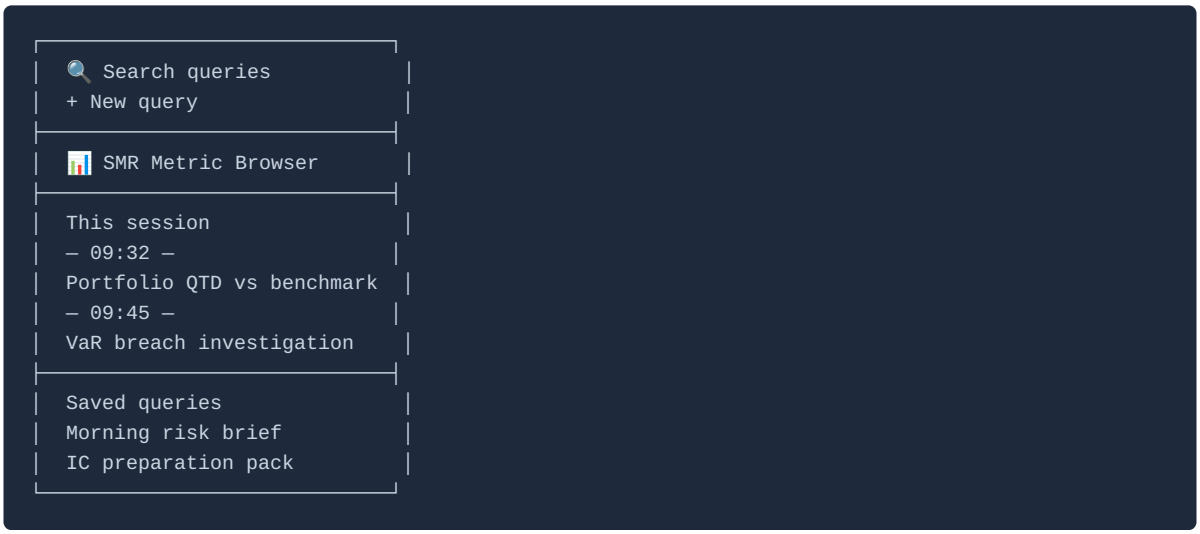
```
import { AiAnalytics } from '@analytics-platform/react';

<AiAnalytics
  tenantId="acme-wealth"
  userToken={jwt}
  onQueryComplete={({ lineageId, metrics }) => handleResult(lineageId, metrics)}
  onGovernanceBlocked={({ circuitBreaker }) => logBlock(circuitBreaker)}
/>
```

Vue:

```
<AiAnalytics
  tenant-id="acme-wealth"
  :user-token="jwt"
  @query-complete="handleResult"
/>
```

Query history panel

The left panel shows all queries from the current session and (optionally) saved queries from prior sessions:



SMR metric browser

Accessible via the left panel, the SMR browser lets users explore registered metrics within their entitlement scope:

- Searchable catalogue of accessible metrics
- Full metric definition: formula (human-readable), unit, aggregation rules, data refresh cadence
- Lineage: upstream sources, downstream metrics
- Owner and steward contact
- Version history (for Power Analysts and above)
- *"Query this metric"* button that pre-populates the query input

Metrics outside the user's entitlement scope are not visible in the browser.

16 — Complementary Services

The AI Analytics Platform assumes the availability of three ecosystem-level complementary services that extend its capabilities for financial services deployments. These services are not owned or operated by the AI Analytics Platform itself — they are shared infrastructure within the broader analytics ecosystem. The platform is designed to integrate with them, but none is a hard dependency for platform operation.

Semantic Registry Service

Overview

The **Semantic Registry Service** is a curated, publicly discoverable library of pre-built metric definitions, dimension schemas, and hierarchy definitions for financial services analytical domains. It reduces the time and expertise required for host teams to populate their Semantic Metrics Registry from scratch.

The Semantic Registry Service is primarily a **config-time resource** — host teams use it when establishing their SMR baseline. Approved definitions may be imported into the tenant's SMR via the Admin API.

What the Semantic Registry Service provides

The service organises its content into analytical domain packages:

Package	Domain	Metric count
`fsi-wealth-v1`	Wealth management and private banking	85 metric definitions
`fsi-investment-v1`	Institutional investment management	120 metric definitions
`fsi-banking-v1`	Retail and wholesale banking	95 metric definitions
`fsi-risk-v1`	Cross-domain risk management	75 metric definitions
`fsi-regulatory-v1`	Regulatory reporting (Basel III/IV, MiFID II, AIFMD)	60 metric definitions
`fsi-esg-v1`	ESG and sustainable investment metrics	45 metric definitions

Each package provides: - **Metric definitions** in the platform's SMR YAML schema (ready for import) - **Dimension schemas** compatible with the Analytics DSL - **Hierarchy definitions** for navigable drilldown - **Measure group collections** for common analytical workflows - **Formula documentation** with calculation methodology references - **Regulatory mapping** where applicable (e.g. which metrics satisfy which regulatory reporting requirements)

Integration with the tenant SMR

Host teams integrate the Semantic Registry Service at initial SMR setup via the Admin API:

```
# Seed the SMR from a registry package
POST /v1/smr/import
{
  "source":  "semantic-registry-service",
  "package": "fsi-wealth-v1",
  "version": "2.3.0",
  "mode":    "merge"   # 'merge' preserves existing definitions; 'overwrite' replaces
}
```

Imported definitions are marked with `source: "semantic_registry_service"` and `source_version: "2.3.0"` in their SMR metadata. When the Semantic Registry Service publishes an updated package version, host teams receive a notification and may import the update via the same API.

Governance of imported definitions

Imported metric definitions are subject to the tenant's normal SMR governance workflow — they enter as `proposed` and require Application Admin approval before becoming resolvable. Application Admins may modify the imported definition before approval if the organisation's calculation methodology differs from the published standard.

Relationship to the platform

The Semantic Registry Service is an optional companion to the platform. Host teams using the `seedTemplate` config field are seeding from a cached snapshot of the relevant Semantic Registry Service package, pre-bundled at platform installation. The live Semantic Registry Service provides the most current definitions and supports packages beyond the core seed templates.

Regulatory Reference Service

Overview

The **Regulatory Reference Service** is an ecosystem service that provides up-to-date regulatory metric definitions, compliance thresholds, and reporting templates for the major financial regulatory regimes. It is a **runtime service** — registered as an execution engine data source that the FQP can route regulatory metric sub-plans to.

Unlike host-managed execution engines, the Regulatory Reference Service holds the authoritative definitions and current threshold values for regulatory metrics — eliminating the need for host teams to maintain regulatory threshold tables internally.

What the Regulatory Reference Service provides

Component	Description
Regulatory metric catalogue	Authoritative definitions for regulatory metrics (LCR, NSFR, leverage ratio, capital ratios, etc.) aligned to current regulatory text

Component	Description
Current threshold values	Regulatory minima and maxima, jurisdiction-specific, updated on regulatory publication schedules
Transition schedule	Announced future threshold changes with effective dates (e.g. Basel III transition to Basel IV)
Reporting templates	Structured result formats compatible with common regulatory submission templates (XBRL, COREP, FINREP)

Registration in the tenant config

```
{
  "executionEngines": [
    {
      "id":           "regulatory-reference",
      "name":         "Regulatory Reference Service",
      "type":         "custom",
      "endpoint":     "https://regulatory.analytics-ecosystem.io/engine",
      "authType":     "api-key",
      "capabilities": ["metric", "filter"],
      "dataAffinity": ["regulatory"],
      "priority":     1,
      "costTier":     "low",
      "enabled":      true
    }
  ]
}
```

When registered and enabled, the FQP routes all sub-plans with `dataAffinity: "regulatory"` to the Regulatory Reference Service first. This ensures regulatory metric values are always sourced from the authoritative service rather than from host-maintained tables that may lag regulatory updates.

Threshold update notifications

When the Regulatory Reference Service publishes updated threshold values (e.g. a jurisdiction's minimum LCR changes), it broadcasts an update notification to all registered tenants via the platform's notification API. The Application Admin reviews the change and (if relevant) updates the tenant's SMR metric display thresholds accordingly.

Relationship to the platform

If the Regulatory Reference Service is unavailable, the FQP falls back to the next registered engine with the `regulatory` data affinity. If no fallback is registered, regulatory sub-plans fail and the user is notified. The platform does not fabricate regulatory threshold values when the authoritative source is unavailable.

Benchmark Data Service

Overview

The **Benchmark Data Service** is an ecosystem service that provides market index and benchmark data as a registered execution engine within the analytics platform. It enables benchmark comparison queries without host teams needing to license, ingest, and maintain benchmark data independently.

What the Benchmark Data Service provides

Data category	Examples
Equity indices	MSCI World, MSCI ACWI, S&P 500, FTSE All-World, Russell 2000
Fixed income indices	Bloomberg Global Aggregate, ICE BofA Investment Grade, JP Morgan EMBI
Multi-asset indices	MSCI ACWI 60/40, AQR Risk Parity
Custom benchmark blends	Host-configurable blended benchmark compositions from registered components
Factor indices	MSCI Minimum Volatility, MSCI Value, MSCI Quality, MSCI Momentum

Registration in the tenant config

```
{
  "executionEngines": [
    {
      "id":           "benchmark-data",
      "name":         "Benchmark Data Service",
      "type":         "custom",
      "endpoint":     "https://benchmarks.analytics-ecosystem.io/engine",
      "authType":     "api-key",
      "capabilities": ["metric", "dimension", "filter", "timeseries"],
      "dataAffinity": ["benchmarks"],
      "priority":     1,
      "costTier":     "low",
      "enabled":      true
    }
  ]
}
```

Benchmark dimension integration

When the Benchmark Data Service is registered, the `benchmark` dimension in the SMR is backed by the service's benchmark catalogue. Users and AI agents can reference benchmark IDs in DSL `COMPARE TO BENCHMARK(...)` clauses, which the FQP resolves against the service.

Custom benchmark blends

Host applications may register custom benchmark blends via the Benchmark Data Service Admin API:

```
{
  "custom_benchmark": {
    "id":          "acme_balanced_benchmark",
    "label":       "Acme Balanced Benchmark",
    "components": [
      { "benchmark_id": "b_msci_world",          "weight": 0.60 },
      { "benchmark_id": "b_bloomberg_global_agg", "weight": 0.35 },
      { "benchmark_id": "b_libor_3m",             "weight": 0.05 }
    ],
    "rebalance_frequency": "monthly"
  }
}
```

Custom benchmarks are accessible within the tenant's queries using the registered `id`.

Data licensing

The Benchmark Data Service operates under data licensing agreements with index providers. Host applications using the service must confirm their licensing entitlement for each index they access. The service enforces licensing checks per tenant — tenants that are not licensed for a specific index cannot access that benchmark via the service.

Complementary service summary

Service	Type	Activation	Primary benefit
Semantic Registry Service	Config-time resource	Import via Admin API	Accelerated SMR setup from vetted financial services metric definitions
Regulatory Reference Service	Runtime execution engine	Register as execution engine in config	Authoritative, up-to-date regulatory metrics without host-side maintenance
Benchmark Data Service	Runtime execution engine	Register as execution engine in config	Licensed benchmark data for comparison queries without host-side data licensing and ingestion

AI Analytics Platform — Roadmap

Current release	v1 — governed semantic analytics layer, SMR, Analytics DSL, FQP, role-aware projection, visualisation ontology, MCP capability layer
Date	May 2026

Near-term — Scheduled and alert-driven queries

Objective: Allow analytical queries to be scheduled for recurring execution and trigger alert workflows when metric thresholds are breached.

Item	Description
Scheduled query registration	Application Admins and Power Analysts register DSL expressions for scheduled execution (cron-based). Scheduled queries run through the full governance pipeline using the owner's entitlement claims at execution time. Results written to the result artefact store.
Threshold alert configuration	Define metric thresholds on scheduled queries. When a result exceeds a threshold, trigger a webhook or in-platform notification.
Alert lineage	Each alert event generates a lineage record linked to the query that triggered it. Alert audit trail is preserved for regulatory review.
Push delivery	Results delivered to registered endpoints (webhook, email, Slack via MCP) with the lineage reference attached.

Pre-requisites: Scheduled execution infrastructure; webhook delivery; alert threshold schema in SMR.

Near-term — Natural language SMR authoring assistance

Objective: Allow metric owners to define new SMR metric definitions in natural language, with the platform generating the structured YAML definition for review.

Item	Description
Natural language → metric draft	Metric owner describes a metric in prose. The platform generates a draft SMR YAML definition with formula, aggregation rules, dimensions, and data domain inferred from the description and validated against the existing SMR.
Consistency check	Generated draft is compared against existing metric definitions to detect potential duplicates, formula inconsistencies, or naming conflicts before the owner reviews.
Review workflow	Generated draft enters the standard SMR approval workflow — it is never directly activated without Application Admin review.

Near-term — Cross-session analytical memory

Objective: Allow users to save analytical context across sessions — preferred dimensions, frequently used metrics, drilldown paths.

Item	Description
Session preferences	User preferences (default dimensions, preferred chart types, favourite measure groups) persisted across sessions. Applied as defaults for new queries.
Saved queries	Named analytical queries saved by Power Analysts and retrievable in future sessions. Saved queries are version-controlled — SMR changes that invalidate a saved query are flagged to the owner.
Personal metric registry	Users can mark metrics from the SMR as "favourites" — surfaced first in intent resolution suggestions and SMR browser.

Phase 2 — Proactive analytical insights

Objective: Surface unsolicited analytical observations when metric values deviate from expected ranges.

Item	Description
Anomaly detection	Background jobs monitor scheduled query results for metric values outside configurable statistical ranges. Anomalies surface as proactive insight cards in the analytical workspace.
Trend signals	Platform identifies metrics showing consistent directional movement across multiple periods. Presented as trend signals with lineage references.
Peer comparison	Where the Benchmark Data Service or Regulatory Reference Service provides peer data, the platform surfaces peer comparison signals.

Pre-requisites: Scheduled query infrastructure; statistical anomaly detection; notification delivery.

Phase 2 — Collaborative analytical sessions

Objective: Allow multiple users within the same tenant to share and collaborate on analytical sessions.

Item	Description
Shared session model	Invite other authenticated users within the tenant to an analytical session. Both participants can submit queries; all results visible to all participants.
Annotation layer	Participants can annotate charts and narrative cards with comments, visible to all session participants.
Session export	Export a complete shared session — all queries, results, narratives, lineage records — as a structured PDF or ZIP archive.

Item	Description
Permission boundary	Participant entitlements are applied individually — each participant's queries are governed by their own role claims. Results from one participant's query are not surfaced to another participant if the other participant would not be entitled to that result.

Phase 2 — Federated drilldown across engines

Objective: Enable drilldown traversal that spans multiple execution engines (currently drilldown is limited to sub-plans served by a single engine per hierarchy level).

Item	Description
Cross-engine drilldown	The FQP coordinates drilldown across engines by identifying the affinity change point in the hierarchy and routing the child-level sub-plan to the appropriate engine. Result assembly handles the cross-engine join at the drilldown boundary.
Drilldown result merging	When a drilldown traversal produces sub-results from multiple engines, the FQP uses the shared dimension keys to assemble a coherent drilldown result.

Phase 2 — Analytical DSL extensions

Objective: Extend the Analytics DSL to support more complex analytical expressions.

Item	Description
`RANK()` and `NTILE()` operations	Ranking metrics within a dimension (e.g. top-10 portfolios by return) expressible natively in DSL. Currently approximated by ordering + limiting.
`WINDOW()` analytics	Moving averages, rolling sums, period-over-period comparisons expressible as DSL window operations rather than separate queries.
`SCENARIO()` modifier	Compare actual metric values against scenario-adjusted values (e.g. stress test scenarios) within a single DSL expression.
`COMPOSITE_BENCHMARK()`	Define blended benchmark compositions inline in DSL without requiring pre-registration in the Benchmark Data Service.

Phase 3 — Headless analytics API

Objective: Expose the full analytical pipeline via a governed headless API for host applications to embed analytical results in their own interfaces without using the web component.

Item	Description
REST analytics API	Full query, result, lineage, and SMR access via authenticated REST API. Same governance pipeline as the web component — no bypass path.
Result streaming	Stream FQP results to the host application in NDJSON format for real-time display in host-owned UI components.
Vega-Lite spec delivery	Return Vega-Lite chart specifications from the Visualisation Ontology as part of the query response — host renders the chart in their own UI using the platform-provided spec.
GraphQL layer	Optional GraphQL API over the headless analytics API for host applications with existing GraphQL infrastructure.

Not in roadmap

Item	Rationale
Raw SQL passthrough	Architecturally prohibited — violates P1 (semantic abstraction). Not planned.
Physical schema exposure	Architecturally prohibited — violates P1. Not planned.
Unauthenticated analytical access	Violates P2 (governance before execution) and P5 (role-aware by default). Not planned.
LLM-selected chart types without ontology	Violates P7 (deterministic visualisation). Not planned.
Cross-tenant result federation	Data confidentiality constraint — tenant boundary is non-configurable. Not planned.

For current product specification, see README.md and the numbered specification documents.